Quant Platform Blueprint

A reference architecture for productionalising hedge-fund quantitative models on Google Cloud

Reference architecture

2026

Contents

Chapter 1

Executive Summary

1.1 What this document describes

A reference architecture for a Software-as-a-Service platform that allows hedge funds to productionalise their own quantitative models — ingesting data, training models, serving inferences, producing audit trails, and delivering outputs back to customer systems — without each customer having to build the surrounding engineering stack.

1.2 Who the target customer is

Systematic and multi-strategy hedge funds, and the quantitative arms of larger asset managers. These organisations employ quant researchers who build models in notebooks and colleagues who struggle to promote those models to production reliably. They are prepared to pay enterprise prices for a platform that closes that gap, and they demand strong data isolation, compliance posture, and integration with their existing identity and data infrastructure.

Public industry research frames the opportunity concretely: industry capital-introduction surveys consistently report that quantitative strategies are among the most-favoured hedge-fund allocations heading into 2026, and that separately managed accounts — structures which require deploying dedicated instances per mandate — are an increasingly common allocation vehicle for institutional investors. Leading quant managers publicly describe their engineering edge in terms of strict research-to-production environments, model governance, and automated golden paths from research to production.

1.3 The architectural bets

1. **Silo tenancy.** Each customer receives a dedicated instance in their own GCP project, or in a customer-owned GCP project under the Bring-Your-Own-Cloud model. Tenant isolation is infrastructure isolation, not a runtime filter. Application code has no notion of tenants.
2. **Local-first development.** Every production behaviour is reproducible on a developer laptop via docker-compose. Services that cannot be simulated locally are disqualified from the architecture. This constraint shapes every other choice.
3. **Single image, multiple roles.** One Docker image built from one codebase is deployed as many services — the API, each projector worker, each pipeline worker, the train-

ing orchestrator, the batch inference worker, the scheduler. Role is selected at startup. Workers autoscale on queue depth independently of the API. Scattered serverless (one function per task) and single-process kitchen-sink (one process with everything as asyncio tasks) are both rejected.

4. **Postgres as the operational substrate.** Event store, work queue, graph store, and time-series store are all Postgres extensions (PGMQ, Apache AGE, TimescaleDB, pg_cron). Blob storage in GCS is the only state outside Postgres. Transactional outbox becomes trivial because the queue shares the database.
5. **Command-Query Responsibility Segregation.** Commands emit versioned events in a single transaction that also writes to state and to queues. Projectors rebuild read models from the event log. The event log is the system of record.
6. **Medallion data platform, file-first.** Bronze (raw files in GCS, content-hashed), silver (validated typed tables in Postgres with bi-temporal lineage), gold (domain aggregates). Inputs and outputs are files by default, matching the reality of hedge-fund integrations.
7. **Research-to-production code parity.** The Python callable used at training time is the same callable used at serving time, packaged into the model artefact. Look-ahead bias and train-serve skew are closed at the pipeline level, not left to researcher discipline.
8. **Customer identity federation.** Customers authenticate through their own Google Workspace or Microsoft Entra directories. The platform issues short-lived session JWTs with application-specific roles, mapped from external claims at login time. No user passwords are stored.
9. **Blue/green deployment via Cloud Run revisions.** Each release is a new revision; traffic shifts in stages; rollback is a traffic-configuration change. Forward-compatible schema migrations make this safe.
10. **Control plane as an internal product.** A separate application operated by the vendor's operations team provisions tenants, orchestrates upgrade waves, and aggregates fleet telemetry. Silo tenancy is operationally viable only because the control plane treats tenant lifecycle as automation rather than manual toil.

1.4 Technology choices, briefly

The Python runtime stack uses the current best-in-class tooling as of 2026: `uv` for packaging, `ruff` for lint and format, `pyright` for types, FastAPI for the web framework, Pydantic v2 for validation, Polars for data frames, Pandera v3 for DataFrame schemas, asyncpg and SQLAlchemy 2 async for the database. The ML stack centres on MLflow 2.x for tracking and registry, the Nixtla suite for time-series forecasting, scikit-learn and gradient-boosted tree libraries for classical ML, PyTorch for deep learning, and Optuna for hyperparameter search. The frontend uses React 19 with Vite, TanStack Query and Router, and shadcn/ui on Tailwind v4. Infrastructure runs on Cloud Run, Cloud SQL, GCS, Secret Manager, and Artifact Registry, provisioned by Terraform and delivered by GitHub Actions with Workload Identity Federation.

1.5 What this document is not

- It is not a product specification. Feature definitions belong in product documentation, not in architecture.

- It is not a timeline. Build sequence is described by prerequisites and acceptance conditions; calendar estimates are omitted because the implementor is agentic and because calendar estimates are the least durable part of any plan.
- It is not a security audit or compliance statement. It describes the posture that supports such audits.
- It is not a replacement for customer-specific engineering. Each customer brings integration requirements that require configuration and, occasionally, extension — the blueprint defines the seams, not the final product for every customer.

1.6 How to read this document

The document is organised as a progression from principles to concrete mechanisms:

- The **Key Ideas** chapter elaborates the ten architectural bets listed above, with enough depth to make them defensible against "why not X?" questions.
- The **Design Brief** through **Roadmap** chapters describe the concrete mechanisms in depth: environment flavours, system and application architecture, tenancy, data and ML platforms, infrastructure, local development, CI/CD, observability, security, and build sequence.
- Technical readers evaluating a specific aspect of the platform can jump directly to the relevant chapter. Decision-makers can read this summary and the Key Ideas chapter and have sufficient grounding to evaluate the architecture as a whole.

Chapter 2

Key Ideas

This chapter elaborates the ten architectural bets named in the executive summary. Each section opens with **what the customer gets** from the bet, then explains the engineering choice that delivers it, the rejected alternative, and the condition under which the decision should be revisited.

The customer-value framing is deliberate. Architectural bets are interesting to engineers; customer outcomes are interesting to buyers. The bets are the *means*; the outcomes are the *ends*. This chapter inverts the usual order so that a non-engineering reader can follow the chapter without losing the thread.

For the customer-facing positioning these outcomes serve, see `blueprint/positioning/2026-04-21-positioning.md`. For how this architecture compares to credible alternatives (SigTech, Domino, Palantir Foundry, Databricks, build-your-own), see Chapter 14.5.

2.1 1. Silo tenancy — infrastructure is the isolation boundary

What the customer gets: Their data and their proprietary code never co-reside with another customer's. For hedge funds whose security teams have rejected multi-tenant SaaS for production-grade workloads, this is the difference between *can deploy* and *cannot deploy*. For LP allocators whose operational due-diligence questionnaires ask about tenancy isolation and data residency, this is a structurally clean answer. For separately-managed-account (SMA) mandates that contractually require per-mandate isolation, it is a deployment shape that fits the legal contract directly.

What we built: Every customer receives a dedicated GCP project containing a dedicated Cloud Run service set, Cloud SQL instance, GCS buckets, and Secret Manager namespace. There is no shared database, no shared application process, no row-level security policy. Tenant separation is achieved by separate deployments, not by runtime filters inside shared infrastructure. Optional Bring-Your-Own-Cloud (BYOC) deployment puts the entire tenant project in the customer's own GCP organisation, with the vendor holding limited-scope deployment access only.

Alternative considered. Pool tenancy with row-level security and a `tenant_id` column on every table is operationally cheaper at low customer count and universally recommended for consumer SaaS. It is the wrong choice here because hedge funds pay for isolation, demand it in security reviews, and sometimes insist that their infrastructure live in their own cloud account. Pool tenancy cannot deliver those properties. SigTech, Domino, and most generic MLOps platforms chose pool; the differentiation case for our platform begins here.

What it costs us. Materially simpler application code (no middleware injecting tenant context, no RLS policies, no noisy-neighbour reasoning) at the cost of a higher per-tenant infrastructure floor and a control plane that must manage a fleet. The control plane is the price of silo; it is a price worth paying for this market segment.

Revisit when: the platform targets a market tier for which silo economics do not hold (sub-thousand-dollar monthly pricing, for instance). Pool tenancy can coexist with silo tenancy within a single codebase, but it doubles the test matrix and is not free.

2.2 2. Local-first development — full production simulation of the *workflow* on a laptop

What the customer gets: A new quant hire is productive in week one, not month three. A researcher iterating on a feature does not wait for cloud round-trips during the inner development loop. A bug surfacing in production can be reproduced on the engineer's laptop within minutes. The "works on my machine" / "fails in CI" / "fails in production" divergence — the bane of mid-sized engineering teams — is structurally eliminated.

The honest scope. Local-first means the **development workflow** is fully reproducible on a laptop, not that production-scale **training compute** runs there. The distinction matters and is sometimes missed:

- **Reproducible locally:** the OIDC authentication flow, the data ingestion pipelines (against scaled-down sample data), the CQRS event loop, the model-training entrypoint (with a small dataset and a small model), the inference serving path, the audit trail mechanics, the integration test suite. The behaviours, not the scales.
- **Not reproducible locally:** training a Transformer on multi-year Alpha360 data, hyperparameter sweeps across hundreds of configurations, GPU-bound deep-learning jobs, multi-day backtest sweeps. These run on managed cloud compute (Cloud Run Jobs for CPU, Vertex AI Custom Training for GPU) by design. The developer launches them with one command from the local environment; the *job* runs in the cloud.

The slogan is *"the same code path runs locally and in the cloud, validated end-to-end"*, not *"production runs on your laptop"*. A team writing pipeline logic against the local docker-compose stack has high confidence that the same logic runs in production; that is the value. They do not (and should not) train a production Transformer on a MacBook.

What we built: A docker-compose stack with Postgres (all extensions), MinIO (GCS substitute), MLflow tracking, and a mock OIDC provider. The application code uses environment variables to select between local and production backends; the major Python libraries in the stack respect well-known emulator conventions (`STORAGE_EMULATOR_HOST`, `MLFLOW_TRACKING_URI`, etc.). Cloud-only services that cannot be simulated locally are disqualified from the architecture. The CI pipeline runs the same docker-compose stack the developers run locally; "works on my machine" diverges from "works in CI" only on resource constraints, not on behavioural differences.

This constraint has shaped multiple downstream choices. Cloud Workflows has no emulator, so orchestration is done with Python state machines. Cloud Tasks has no emulator, so queueing is done with PGMQ. Cloud Scheduler has no emulator, so in-application APScheduler does the job. Identity Platform is rejected in favour of direct OIDC federation because the blocking-function model fragments code across Cloud Functions.

Alternative considered. Cloud-tethered development (the Databricks / SigTech / Domino model) where the researcher works against a hosted environment from day one. Faster on-

boarding to a *running* environment; slower inner-loop iteration; harder to debug; harder to test changes; harder to develop offline. The trade-off favours local-first for our segment.

Revisit when: never. This is the governing constraint for the development workflow.

2.3 3. Single image, multiple roles

What the customer gets: One codebase to reason about, one test suite to maintain, one image to scan for vulnerabilities, one CI/CD flow to operate — but with the runtime independence of microservices. A worker pool can scale on queue depth without affecting the API; a misbehaving worker cannot starve the API of event-loop time; release cadence is per-codebase, not per-service. The customer's engineering team operates a small fleet of single-purpose services without paying the operational overhead of a microservice sprawl.

What we built: One Docker image is built from one codebase and deployed as many services, each running a different role. Roles include: the API server, each projector worker, each pipeline worker, the training orchestrator, the batch inference worker, and the scheduler daemon. At container startup, a single environment variable selects which role the process plays; the rest of the image is identical.

This is distinct from two patterns sometimes conflated with it and both rejected:

Scattered serverless — one Cloud Function or Cloud Run service per task, typically with its own repository or its own CI/CD pipeline. This fragments the codebase, multiplies operational surface area, breaks local testability, and loses the transactional guarantees that come from a shared database client. A team running thirty such services is slower to diagnose, slower to refactor, and more prone to version-skew bugs than a team running one codebase.

Single-process kitchen sink — one `uvicorn` process hosting the API plus every worker type as asyncio tasks. This is simpler for a toy prototype but fails in production: workers cannot scale independently of the API, a misbehaving worker can starve the API of event-loop time, and queue backlog pressure cannot drive additional worker capacity because there is no worker capacity to add. The single-process pattern is rejected on the same grounds as scattered serverless: it sacrifices the runtime independence that distinguishes a production worker pool from a prototype.

The single-image-many-roles pattern avoids both failure modes. Workers must autoscale on queue depth, not just on CPU. A graph projector pool with ten thousand pending messages must scale out even if individual worker CPU is low. A pool with a drained queue must scale back in to zero or one replica. The infrastructure chapter describes the three mechanisms by which this queue-depth autoscaling is implemented on GCP.

Revisit when: per-role release cadences diverge enough that the API and the workers can no longer share a build train, or one role's dependency footprint must shrink dramatically (typically for cold-start reasons) and cannot do so within a shared image. The specific per-role deployment target (Cloud Run versus GKE) may evolve based on scaling requirements; the single-image-many-roles property is stable across either choice.

2.4 4. Postgres as the operational substrate

What the customer gets: Transactional coherence across event store, aggregate state, message queue, graph store, time-series store, scheduler, and audit log. The "transactional outbox" pattern — a recurring source of subtle distributed-systems bugs — simply ceases to exist

as a problem because the queue lives in the same database as the state changes it announces. For the customer's compliance team, the audit log is *queryable in the same database* as the operational state; for the customer's quant team, a command handler that updates aggregate state and enqueues projection messages does so in one transaction that either commits or fails atomically.

What we built: A single Postgres database — Cloud SQL by default, AlloyDB where required — holds the event store, the aggregate state, the message queue (PGMQ), the graph store (Apache AGE), the time-series store (TimescaleDB), the scheduler (pg_cron), the user registry, the audit log, and the read models. Blob storage in GCS holds raw files and model artefacts; everything else is in Postgres.

The driving insight is that most architectural services the industry typically implements as separate products — Kafka for queueing, Neo4j for graphs, Kdb or InfluxDB for time series, Redis for caching — have first-class Postgres extensions that cover the same ground at a scale many times larger than the target customer's workload. Treating Postgres as the platform eliminates cross-system consistency problems, collapses the operational surface, and makes everything transactionally coherent.

Alternative considered. Multi-store architectures (Kafka + Postgres + Neo4j + Kdb + Redis) are the default in the modern data-platform world, particularly at the petabyte scale Databricks targets. They are over-engineered for a customer whose operational data fits comfortably in Postgres (which is essentially every quant shop with under $20B AUM) and they multiply the failure domains. We deliberately chose simpler.

Revisit when: a specific extension hits a fundamental ceiling. PGMQ handles tens of thousands of messages per second on commodity hardware (per Tembo's published PGMQ benchmarks); beyond that, a dedicated broker (Kafka) may be required. AGE's traversal planner is fine for shallow-graph queries; deep graph analytics at scale may demand Neo4j. TimescaleDB covers most time-series needs; tick-level HFT data may demand Kdb. None of these conditions apply to the target customer segment today.

2.5 5. Command-Query Responsibility Segregation

What the customer gets: A complete, replayable history of every state change, queryable forever. The compliance officer can answer "what did the system know at 4pm on the trade date" by replaying the event log. The quant lead debugging an unexpected inference output can trace it back through every command that produced it. The LP auditor asking "show me how this strategy's positions evolved over the past month" sees an answer derived from immutable events, not from interpreted state. CQRS sounds like an engineering pattern; for the customer it is the substrate that makes audit, reproducibility, and time-travel queries possible at all.

What we built: Writes go through aggregates that emit versioned events. Reads come from purpose-built projections, each populated by a projector that consumes events from a PGMQ queue. The event log is the source of truth; everything else is derived.

CQRS is sometimes seen as a complexity burden imposed on simple systems. The choice here is deliberate and contextual. Hedge-fund workloads are audit-heavy, regulator-visible, and demand point-in-time reconstruction of past states — exactly the conditions where the event log becomes load-bearing.

The specific CQRS shape chosen has three properties worth noting. First, events are persisted in the same Postgres as aggregate state, enabling the single-transaction guarantee described

in §4. Second, each projector role runs as a dedicated worker service stamped from the same image, with internal concurrency implemented via asyncio tasks within that worker; projector workers are not co-resident with the API process but share its codebase, test suite, and build pipeline. Third, every projector is idempotent by construction, keyed on `event_id`; rebuilding a projection is a matter of truncating its state table and replaying the event log.

Revisit when: a specific workload does not benefit from the event-sourced model — a short-lived internal tool, a pure analytics dashboard. CQRS is the default for the customer-facing platform, not a universal rule.

2.6 6. Medallion data platform, file-first

What the customer gets: The platform's data ingestion shape *fits* the actual integration reality of hedge-fund customers, who exchange files with vendors, prime brokers, custodians, and fund administrators rather than streaming events. A data engineer at a customer integrating a new vendor does not have to convince that vendor to build a streaming integration; they configure a file pattern. Streaming, where it exists, is treated as a special case of micro-batched files, which means the rest of the data platform's machinery applies uniformly.

The other gift to the customer is **point-in-time correctness as a schema property**: every silver and gold row carries a `_knowable_at` column (system time — when the datum became visible to the platform) alongside `_valid_from` and `_valid_to` columns (the business-time interval over which the datum applies). Training-data extraction queries without a `_knowable_at` filter fail validation at pipeline-build time; bi-temporal reconstructions ("what did we believe on date T about period P?") combine both halves. The data platform chapter documents the four columns in full. This is the discipline that prevents look-ahead bias from being a researcher discretionary practice and makes it instead a structural property.

What we built: Inbound data arrives as files. Outbound data leaves as files. Between them, three layers of progressively more curated data live in GCS (bronze, immutable parquet) and Postgres (silver and gold, typed and aggregated). Every bronze file is identified by content hash. Every silver row carries lineage to its bronze source. Every gold aggregate is reproducible from silver by re-running a pure function.

The medallion layers are wrapped in Dagster's software-defined-asset model: bronze, silver, and gold tables are assets, dependencies between them are the asset graph, and data-quality rules are asset checks attached to the asset they validate. The customer's quant lead sees the lineage of any gold aggregate back to its bronze sources in one screen; the customer's compliance officer sees data-quality breaches at the same place they see the asset itself; the LP allocator running operational due diligence sees concrete evidence that gold-layer outputs trace cleanly back to vendor files with no off-graph manual steps. APScheduler triggers materialization runs on cadence, PGMQ carries the event-driven materialization triggers, and Dagster owns the asset graph; the three coexist rather than compete.

Alternative considered. Streaming-first / event-driven data platforms are the default in cloud-native architectures (Kafka Flink object store). They are well-suited to ad-tech, IoT, and real-time analytics workloads. For hedge-fund integrations they are over-engineered: vendors send files, regulators expect file evidence, audit requires file provenance. Designing the platform around file exchange and treating streaming as a file-micro-batching special case yields a system that fits its integration surface.

Revisit when: a specific customer's workload is genuinely streaming-first (market-making, sub-second signal generation). Extend the platform to run streaming as the first-class path for that customer, not the other way round.

2.7 7. Research-to-production code parity

What the customer gets: The single most expensive class of bug at quant shops — the feature that worked in the research notebook but produces different numbers in production — ceases to exist as a class. The quant who finishes a research piece does not hand off to engineering for re-implementation; the research code *is* the production code. Train-serve skew is closed at the platform level, not left to team discipline.

What we built: The Python function that computes features at training time is the same function that computes features at serving time. It is packaged into the MLflow `pyfunc` artefact alongside the trained model, so that the model and its preprocessing travel as one unit. Researchers import the same function when prototyping; the notebook becomes a draft of the production serving path, not a parallel artefact that will later be re-implemented.

This addresses the second-largest class of quantitative-strategy failures, after look-ahead bias: code divergence between research and production. A feature computed one way in the notebook and a different way in production silently breaks the model. It is a class of error that quantitative teams rediscover repeatedly, and the only reliable fix is to make the code identical by construction.

The `pyfunc` wrapper is the vehicle. It captures both the trained weights and the inference code. Upgrading a model cannot break the serving endpoint's calling contract, because the endpoint calls the wrapper, not a hand-written preprocessor.

Revisit when: the model registry's `pyfunc` packaging convention ceases to be the standard vehicle for the libraries the platform depends on, or a customer's preferred framework cannot be wrapped without losing the parity guarantee. Legacy models without `pyfunc` wrappers may ship through a different path during migration; the target is zero such models in steady state.

2.8 8. Point-in-time correctness as a platform property

What the customer gets: Backtests that survive their own promotion to production. A model trained on data filtered by `_knowable_at <= :as_of` is trained on a counterfactual the manager could have actually constructed at the simulated decision time; the alpha that shows in backtest is alpha that would have been available in production. For an LP allocator running ODD, a manager whose platform enforces this discipline as a query-time gate (not a researcher's optional practice) is a manager with a structurally honest backtest pipeline.

What we built: The platform maintains both system-time and valid-time on silver and gold rows: a `_knowable_at` column records when the datum first became visible to the platform; `_valid_from` and `_valid_to` record the business-time interval over which the datum applies. Both are distinct from the original business-event timestamp. Queries that extract training data must filter by `_knowable_at <= :as_of`; queries without this filter fail pipeline validation before they ever run. Temporal queries that ask "what did we believe at time T about period P?" combine the two: filter `_knowable_at <= T` and intersect with valid-time on P.

This is the primary defence against look-ahead bias, which the quantitative-finance literature consistently identifies as a leading cause of strategies that appear profitable in backtests and fail in production (López de Prado, *Advances in Financial Machine Learning*, Wiley 2018, Ch. 11–13; Bailey & López de Prado, "The Probability of Backtest Overfitting," *Journal of Computational Finance*, 2014). The typical scenario is innocuous-looking: a consumer credit-card transaction dated Monday is aggregated by the vendor on Wednesday and arrives at the cus-

tomer on Thursday. Using Thursday's file to backtest decisions as of Monday introduces days of hindsight into the simulation and produces spurious profits.

Bi-temporal data models are well understood in the academic literature and in specialist financial-data vendors' products. They are rarer in general-purpose data platforms, because they add modelling overhead. The blueprint takes the overhead as non-negotiable for the target market.

Revisit when: an as-yet-unforeseen workload demonstrates that the bi-temporal modelling overhead exceeds its benefit — none anticipated for the target customer segment.

2.9 9. Blue/green deployment via Cloud Run revisions

What the customer gets: Releases that ship without downtime, with a one-command rollback path. The customer's quant lead can promote a new model on a Friday afternoon, have it serve a fraction of traffic for the weekend, and either roll out or roll back on Monday based on observed metrics — without any of this requiring engineering intervention. For the customer's compliance team, every release is a discrete event with its own audit trail entry; a "what changed when" question has a structured answer.

What we built: Every release is a new Cloud Run revision. Traffic to the new revision starts at zero and shifts in configured increments with health checks between each step. The previous revision is retained for a configured window to allow immediate rollback via a single traffic-configuration change.

Cloud Run's native revision model is the blue/green primitive. There is no need for a separate blue/green system, no need for traffic-management infrastructure, no need for container orchestration outside what Cloud Run provides. The rollback path is a command, not a rebuild.

The property that makes this safe is forward-compatible schema migrations. The new revision must work with both the old schema (before migration) and the new schema (after migration). The expand-migrate-contract pattern enforces this: add columns and tables (safe for both versions), deploy new code that uses them, backfill as needed, remove the old structures in a later release. This adds a release cycle to every schema change but eliminates a class of downtime incidents.

Revisit when: a specific release requires a breaking change that cannot be made forward-compatible. Such releases are rare and should be planned carefully — typically across multiple quieter releases — rather than forced into one deployment.

2.10 10. Control plane as an internal product

What the customer gets: Indirectly, what the customer gets from the control plane is *operational reliability of the silo tenancy model itself*. A vendor without a control plane manages a fleet of silo tenants by hand; the per-tenant operational mistakes compound; eventually the customer notices that an upgrade landed late, a backup didn't run, a maintenance window was missed. The control plane is the price the vendor pays so that silo tenancy remains operationally credible at scale.

For the user (vendor operator), the control plane is the tool that makes managing fifty tenants feel like managing one. Provisioning, upgrading, monitoring, and billing are all driven through it.

What we built: The control plane is a separate application, run by the vendor's operations team, that manages the fleet of tenant instances. It knows every tenant, their current version, their maintenance window, their contract tier. It orchestrates provisioning (Terraform apply), upgrades (wave-based rollouts), and fleet telemetry. Customers never see the control plane.

The existence of a control plane is the operational consequence of silo tenancy. A single-tenant SaaS business has no use for a control plane; a hundred-tenant silo SaaS business lives or dies by the quality of its control plane. Treating it as a first-class product — with its own release cadence, observability, and success metrics — is what makes silo economics work at scale.

The control plane shares the repository and CI/CD pipeline of the main application, because the two products evolve together: a new application feature that requires a new Secret Manager entry must be reflected in the provisioning flow for new tenants. Decoupling the repositories would mean every application change requires a matching control-plane change in a separate PR, which is friction without benefit.

Revisit when: the control plane's complexity outgrows the team's ability to evolve it in-line with application changes. Splitting it out then is a deliberate graduation, not an early optimisation.

2.11 Summary

The ten bets are mutually reinforcing. Silo tenancy demands a control plane; local-first constrains technology choices; Postgres-centric design enables transactional CQRS; CQRS plus bi-temporal data delivers point-in-time correctness; file-first medallion data matches the customer's integration reality; code parity plus the registry eliminate train-serve skew; Cloud Run revisions plus forward-compatible migrations deliver safe continuous delivery; customer IdP federation keeps the platform out of the user-store business.

Read in customer-value terms: silo + BYOC says *your data and code stay in your cloud*; local-first says *your developers iterate on a laptop, not a Databricks workspace*; single-image-multi-role + CQRS + Postgres-centric say *your operations team manages a small comprehensible fleet, not a distributed-systems Rube Goldberg*; medallion + bi-temporal say *your backtests are honest by construction*; research-to-production parity says *your quants ship without an engineering hand-off*; blue/green + control plane say *your platform stays available and evolves under wave-based releases without your team owning that orchestration*.

The detailed chapters that follow describe how each of these ideas is implemented concretely. Chapter 14.5 compares this architecture explicitly to the credible alternatives.

Chapter 3

Design Brief

3.1 What this blueprint is

A reference architecture for building a multi-tenant Software-as-a-Service platform that enables quantitative hedge funds to productionalise their trading models. It covers the end-to-end stack: a React user interface, a Python application exposing a REST API, authenticated against customer-managed enterprise identity providers, a data ingestion and transformation platform, a model training environment, a model serving surface, and the supporting infrastructure, deployment, and operational tooling.

The blueprint is opinionated. It favours a small number of well-understood components over a large number of novel ones, and it requires that the entire system can be run and exercised on a developer laptop in a faithful simulation of production.

3.2 Who this platform is for

Quantitative investment firms — systematic hedge funds, multi-strategy shops, and quantitative arms of larger asset managers — whose internal teams build predictive, risk, or execution models but lack the engineering substrate to move those models from research notebooks to production-grade services. The platform provides the engineering foundation these customers cannot or will not build in-house, while ceding to them full control over their data, identity, and deployment environment.

Current industry research makes this market framing concrete. Equity market neutral, quant multi-strategy, and global macro have been among the most-favoured quantitative strategies in successive industry capital-introduction surveys entering 2026. Separately managed accounts — structures in which a single institutional investor receives a dedicated, customised implementation of a manager's strategy — are a substantial and growing share of how managers serve large allocators, and they push managers towards deployment patterns that can stamp out dedicated instances per mandate. Leading quantitative managers publicly describe their engineering edge in terms of model governance (versioned inventories of models with documented inputs, signals, and validation artefacts) and strict separation between research, development, and production environments; published accounts of internal developer platforms at large quant managers describe model-to-production workflows reduced from weeks of manual handover to minutes through a configuration-driven golden path. The architectural assumption in this blueprint — that each customer receives a dedicated silo that their own quants can push models into through a standardised, audited workflow — aligns directly with where this segment is moving.

Typical users within a customer organisation fall into four personas:

- **Quant developers** who train, version, and promote models
- **Risk and compliance officers** who inspect model behaviour and audit trails
- **Operations engineers** who monitor data pipelines, model performance, and infrastructure
- **Platform administrators** who manage users, roles, and tenant configuration

3.3 Goals

1. **Faithful local simulation of the production** *workflow***.** Every production *behaviour* is reproducible on a single developer machine via docker-compose — auth flow, ingestion, CQRS event loop, model-training entrypoint (small data + small model), inference serving, audit. The slogan is "the same code path runs locally and in the cloud, validated end-to-end," not "production runs on your laptop." Production-scale training compute (deep-learning jobs, hyperparameter sweeps, GPU-bound work) explicitly runs on managed cloud (Cloud Run Jobs, Vertex AI Custom Training); the developer launches it with one command from the local environment, the *job* runs in the cloud. This is the governing constraint for the development workflow; every architectural decision is evaluated against it.
2. **Silo tenancy.** Each customer receives a dedicated instance of the platform, either in the vendor's GCP organisation or the customer's own (BYOC). Data never crosses tenant boundaries.
3. **Enterprise identity federation.** Customers bring their existing Google Workspace or Microsoft Entra directories. The platform does not maintain its own user store.
4. **Portable open-source components.** Where a managed cloud service and a portable open-source component are both viable, the open-source component wins unless the cost of running it is unjustifiable.
5. **Postgres-centric data plane.** Operational state, event sourcing, message queueing, time-series data, and graph data all live in Postgres via extensions. Blob storage for raw artefacts is the only exception.
6. **Near-monolithic application.** One codebase, one Docker image, one test suite, one mental model — deployed as a small set of single-purpose services (the API, projectors, pipeline workers, training and inference workers, the scheduler) selected at startup by an environment variable. Workers scale independently of the API, but share the build, test, and release pipeline.
7. **Continuous delivery with safe rollouts.** Every change is built, tested, and promoted through staged environments with blue/green traffic splitting and instant rollback.
8. **Per-tenant operability.** Each instance is independently observable, upgradable, and billable. Fleet operations scale to hundreds of tenants through a thin control plane, not through per-tenant heroics.

3.4 Design principles

Principle	Implication
Local-first	Any service added to production must have a local equivalent; otherwise it is not used
Less is more	One Postgres with extensions, one application process, one CI/CD pipeline

Principle	Implication
Boring technology	Proven, widely-supported components with strong communities and documentation
Event-sourced by default	State is derived from an append-only event log; read models are projections
Silo over pool	Tenant isolation comes from separate deployments, not row-level filters
Forward-compatible migrations	Schema changes are additive; blue/green deploys never require synchronous migrations
Code is the contract	Event schemas, API contracts, and infrastructure live in version control and are tested

3.5 Non-goals

- Freemium, self-serve, or small-business tiers. The floor cost of silo tenancy makes these economically unviable.
- A managed research environment. The platform productionalises models; it does not replace Jupyter, notebooks, or research clusters.
- A trade execution venue, order management system, or custody solution. The platform consumes and produces data; it does not route orders or hold positions.
- A general-purpose machine learning platform. The architecture is tuned for quantitative finance workloads: point-in-time correctness, deterministic reproduction, audit traceability.
- Real-time algorithmic trading with sub-millisecond latency requirements. The serving model targets batch and near-real-time inference; ultra-low-latency paths require specialised infrastructure outside this blueprint.

3.6 Success criteria

A new engineer clones the repository, runs a single command, and has a working local instance with seeded data in under fifteen minutes. The integration test suite runs the full stack without touching cloud resources. A new tenant is provisioned from a single operator action and is live within thirty minutes. A release candidate is promoted to production with a blue/green traffic shift that can be rolled back in seconds. A model trained by a customer on Monday is running in production inference by Tuesday with a full audit trail linking the inference output back to the training data.

Chapter 4

Environment Flavours

The platform runs in five environment profiles, which differ in their deployment topology, data sources, and operational posture but share a single codebase and a single container image.

4.1 Local

A full production simulation running on a developer laptop via docker-compose. Postgres with AGE, TimescaleDB, PGMQ, and pg_cron extensions. MinIO for blob storage. A mock OIDC provider substituting for customer identity providers. MLflow for model tracking. The application itself running under uvicorn with hot reload. The React UI served by Vite's dev server or by the application's static-serving middleware.

All code paths that exist in production execute locally. External data feeds are replaced by seed fixtures loaded from parquet files. Secrets come from a local `.env` file. Logs stream to stdout in JSON format, the same format used in production.

A new engineer runs `make dev` to bring up the stack, `make migrate` to apply schema changes, `make seed` to populate test tenants and users, and `make test` to run the full integration suite. The entire cycle completes without a network connection to GCP.

4.2 Shared development sandbox

A shared GCP project containing a single instance of the platform used by the engineering team for exploratory testing, demos, and pre-merge integration. It mirrors production topology but uses smaller instance sizes, lower retention, and anonymised seed data. Access is via the team's Google Workspace; there is no per-tenant customer isolation.

The sandbox is not intended for customer data, customer demos, or performance testing. It exists to catch integration issues that the local stack cannot reproduce — real Cloud SQL behaviour, real Artifact Registry image pulls, real Workload Identity Federation tokens.

4.3 Staging

A per-tenant replica of production, deployed in the vendor's GCP organisation, used for user acceptance testing before a customer promotes a release to their production silo. Identical

configuration to the customer's production instance but pointed at the customer's UAT identity provider (or a dedicated staging realm within their corporate directory) and loaded with anonymised or synthetic data.

Staging exists for enterprise-tier customers. Standard-tier customers skip directly from the vendor's internal staging to their own production silo.

4.4 Production silo

A dedicated instance of the platform per customer, deployed in the vendor's GCP organisation, isolated in its own GCP project. Contains a Cloud Run service (the application), a Cloud SQL Postgres instance with the full extension suite, a GCS bucket for blob storage, an MLflow tracking server, Secret Manager entries for the customer's identity provider configuration, and the customer's custom domain bound through Cloud Load Balancing.

The vendor operates the instance; the customer consumes it through their browser and through scheduled integrations pushing data into GCS. Lifecycle — provisioning, upgrades, monitoring, incident response — is managed by the vendor's control plane.

4.5 Production BYOC

The same architecture as production silo, but deployed into a GCP project owned by the customer. The vendor retains a limited-scope service account with permissions to deploy new revisions, read telemetry, and run upgrade migrations. All data, including backups, lives within the customer's cloud perimeter.

BYOC is the deployment model of choice for customers with strict data residency obligations, regulatory oversight requiring on-premises equivalence, or internal policies prohibiting data egress to vendor-controlled environments. It is operationally heavier — the customer's change-management process applies to every release — but is often the only viable model for the largest customers.

4.6 Comparison matrix

Aspect	Local	Sandbox	Staging	Prod Silo	Prod BYOC
Hosting	Laptop	Vendor GCP	Vendor GCP	Vendor GCP	Customer GCP
Isolation	Single-tenant	Shared	Per-customer	Per-customer	Per-customer
Identity	Mock OIDC	Vendor Workspace	Customer UAT	Customer prod	Customer prod
Data	Seed fixtures	Anonymised	UAT data	Production	Production
Cloud SQL size	Local container	db-f1-micro	db-custom-2-7680	tier-appropriate	tier-appropriate
Min instances	0	0	1	1	1
Change cadence	Every commit	Every PR	Weekly	On customer approval	On customer approval

Aspect	Local	Sandbox	Staging	Prod Silo	Prod BYOC
Observability	stdout	Shared dashboard	Per-tenant dashboard	Per-tenant dashboard	Per-tenant dashboard (often mirrored to customer)
Cost ownership	Vendor	Vendor	Vendor	Vendor	Customer

4.7 Environment selection at runtime

The application ships as a single container image. Environment differences are expressed entirely through:

- **Environment variables** resolving service endpoints (`DATABASE_URL`, `STORAGE_BACKEND`, `OIDC_PROVIDER_URL`, `MLFLOW_TRACKING_URI`)
- **Mounted secrets** from the local `.env` or from Secret Manager in the cloud
- **Feature flags** evaluated at startup for environment-specific behaviour (synthetic data injection, verbose logging, test endpoints)

No environment-specific code paths exist. A bug reproducible in production is reproducible locally by loading the same configuration.

Chapter 5

System Architecture

5.1 Context

The platform sits between three classes of external system: the customer's enterprise identity provider (Google Workspace or Microsoft Entra), the customer's market and reference data sources (delivered via SFTP, API, or object storage hand-off), and the vendor's operational substrate (GitHub, Google Cloud, and a thin internal control plane used by vendor operations staff).

Users interact with the platform exclusively through a React single-page application served by the application container. All user-facing operations traverse a single REST API; there is no direct customer access to the database, the blob store, or the underlying GCP resources.

5.2 Container view

A single customer instance consists of five deployed components:

1. **The application image, deployed as a small set of single-purpose services** — one Docker image stamped from the same codebase, deployed as a set of Cloud Run services that differ only in the `ROLE` environment variable they read at startup: the `api` (REST handlers, React static assets, command dispatch, OIDC callbacks), one service per projector role (`worker-proj-ui`, `worker-proj-graph`, `worker-proj-analytics`), one per pipeline worker role (`worker-pipeline-bronze`, `worker-pipeline-silver`), the `worker-training` job dispatcher, the `worker-inference-batch` processor, and the `scheduler` daemon. Every service runs the same image and shares the same database connection patterns; they differ only in role, autoscaling policy, and IAM bindings. The `api` service runs with a minimum of one always-warm instance; worker services autoscale on queue depth (see Application Architecture chapter for the full role table). The full role table and rationale are in the Application chapter.
2. **Postgres (Cloud SQL by default; AlloyDB where required)** — the operational data store, event log, queue, graph, and time-series backend. Configured with the AGE, TimescaleDB, PGMQ, and pg_cron extensions. Cloud SQL is the default; AlloyDB is required only if AGE or PGMQ are not available on Cloud SQL's supported extension list in the deployment region. The Infrastructure and Application chapters reference this rule rather than restating it.
3. **Object storage (GCS)** — raw ingested files (bronze layer), model artefacts, training datasets, and inference batch outputs. Accessed via a storage abstraction that binds to

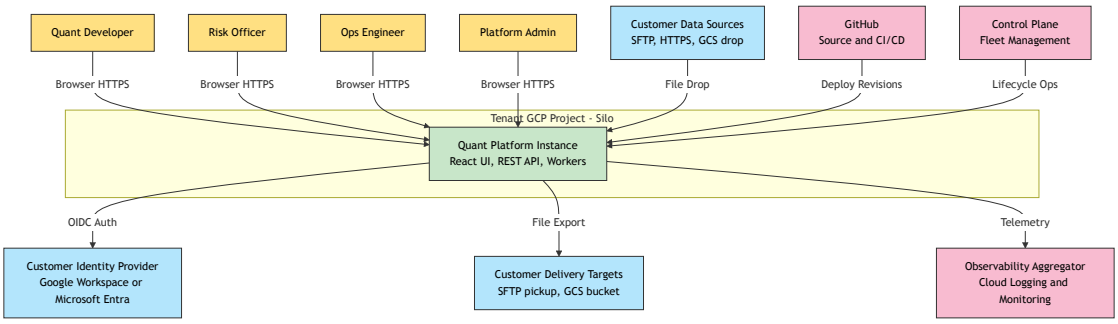

Figure 5.1: System context

MinIO in local development.

4. **MLflow tracking server** — model experiment tracking, run metadata, and model registry. Backed by the same Postgres instance (separate schema) and uses GCS as the artefact store. Deployed as a separate Cloud Run service to isolate its upgrade cadence from the main application.
5. **Static frontend** — the React SPA, served either from the application container (simpler, adequate for moderate traffic) or from a CDN-fronted GCS bucket (lower latency, preferred for production). Both paths are supported by the same build artefact.

5.3 Components within the application

The application container is logically partitioned into modules that share a single Python process and a single database connection pool but maintain clear boundaries at the code level.

- **API surface** — REST handlers for user-facing commands and queries, plus internal webhook endpoints that receive PGMQ push deliveries and OIDC callbacks.
- **Domain** — pure Python business logic: aggregates, commands, events, invariants. No I/O; no framework dependencies.
- **Projections** — handlers that consume events from PGMQ queues and update read models in Postgres tables, AGE graphs, and TimescaleDB hypertables.
- **Pipelines** — ingestion and transformation functions organised in medallion layers, invoked by the scheduler or by event triggers.
- **Training** — model training orchestration that coordinates data extraction, training job dispatch, artefact persistence, and model registration.
- **Serving** — model inference endpoints that load versioned models from the MLflow registry and expose them through the REST API.
- **Infrastructure adapters** — database sessions, PGMQ client, blob store client, MLflow client, identity provider client. All pluggable by environment.

5.4 Technology stack summary

Technology choices reflect current best-in-class options across the Python and data-engineering ecosystem as of 2026. Where a newer option has meaningfully displaced an older default (uv replacing pip/poetry, Polars replacing pandas, Nixtla replacing ad-hoc time-series code), the newer option is preferred.

5.4.1 Application runtime

Concern	Component	Rationale
Web framework	**FastAPI**	Async-native, Pydantic-v2 integrated, first-class OpenAPI; widely adopted across the Python ecosystem. Ecosystem dominance over Litestar discussed below in “Why these choices over common alternatives”

Concern	Component	Rationale
ASGI server	**uvicorn** (with `--workers` in production)	Reference ASGI server; Granian is a contender but uvicorn remains the default
Validation	**Pydantic v2**	Rust-backed validators, schema generation, FastAPI integration
Data-frame validation	**Pandera v3**	Schema-first DataFrame validation; first-class Polars support
Data frames	**Polars**	Rust-backed, lazy evaluation; on the bulk transformation workloads typical of this platform (silver/gold group-bys and joins on multi-million-row frames), order-of-magnitude faster than pandas with materially better memory characteristics (per the Polars project's published benchmarks at pola.rs/benchmarks). Pandas retained at integration points with libraries that require it
In-memory analytics	**DuckDB** (optional)	Columnar analytics engine; Arrow-native interop with Polars. Used where complex analytical queries against in-memory datasets outperform round-trips to Postgres
HTTP client	**httpx**	Async-native, drop-in replacement for requests
Task scheduling	**APScheduler**	In-process cron; durable via Postgres job store. Triggers Dagster materialization runs on schedule and handles the cron cases that do not warrant an asset

Concern	Component	Rationale
Pipeline orchestration	**Dagster**	Software-defined-asset model, lineage, asset checks, and a visual DAG UI. Run storage reuses the existing Postgres instance; deployed as a docker-compose service locally and a Cloud Run service in production. The UI is proxied read-only via the BFF under `/dagster/*`

5.4.2 Data platform

Concern	Component	Rationale
Database	**Postgres 16**	Foundation for all operational data; extensions cover specialised needs
Graph	**Apache AGE** extension	Cypher inside Postgres; no separate graph database
Time series	**TimescaleDB** extension	Hypertables, compression, continuous aggregates inside Postgres
Message queue	**PGMQ** extension	Transactional with state changes; no separate broker
DB scheduler	**pg_cron** extension	In-database cron for maintenance and materialised-view refresh
DB driver	**asyncpg** (direct) + **SQLAlchemy 2.x async** (ORM)	Raw async for hot paths, ORM for CRUD and migrations
Migrations	**Alembic**	Standard; supports async SQLAlchemy
Blob storage	**GCS** (prod), **MinIO** (local)	S3-compatible; abstracted by application adapter
Object client	**google-cloud-storage** (respects `STORAGE_EMULATOR_HOST`)	Single client for both environments

5.4.3 ML platform

Concern	Component	Rationale
Experiment tracking & registry	**MLflow 2.x**	Open-source, Postgres-backed, full local stack

Concern	Component	Rationale
Model packaging	**MLflow `pyfunc`**	Captures serialisation + inference wrapper in one artefact
Classical ML	**scikit-learn**, **XGBoost**, **LightGBM**, **CatBoost**	Best-in-class for tabular quantitative workloads
Time-series forecasting	**Nixtla suite** (StatsForecast, MLForecast, NeuralForecast, HierarchicalForecast), **Darts**	Leading Python ecosystem for classical, ML, and deep time-series; hierarchical reconciliation first-class
Foundation models for TS	**TimeGPT** (optional)	Pre-trained time-series foundation model; useful for cold-start scenarios
Deep learning	**PyTorch 2.x** (default) or **JAX** (research-led teams)	PyTorch for production; JAX where the customer's research team is already there
Backtesting	**vectorbt** / **vectorbtpro**, **QSTrader**, **LEAN**	Current leading vectorised and event-driven frameworks; choice driven by strategy shape
Quant numerics	**NumPy**, **SciPy**, **QuantLib** (where derivatives pricing applies)	Standard foundation
Hyperparameter search	**Optuna**	Async-friendly, MLflow-integrated, current default

5.4.4 Frontend

Concern	Component	Rationale
Framework	**React 19**	Industry standard; server components optional for SSR variants
Build tool	**Vite**	Dominant dev-server and build pipeline
Router	**TanStack Router**	Type-safe, preferred default for new React projects in 2026
Data fetching	**TanStack Query**	Cache, mutations, background refetch; current default for server state
UI primitives	**shadcn/ui** (Radix + Tailwind v4)	Copy-paste components; direct ownership of UI code
Forms	**react-hook-form** + **zod**	Typed, performant forms with schema validation
API client	Generated from OpenAPI via **openapi-typescript**	Zero drift between server and client contracts

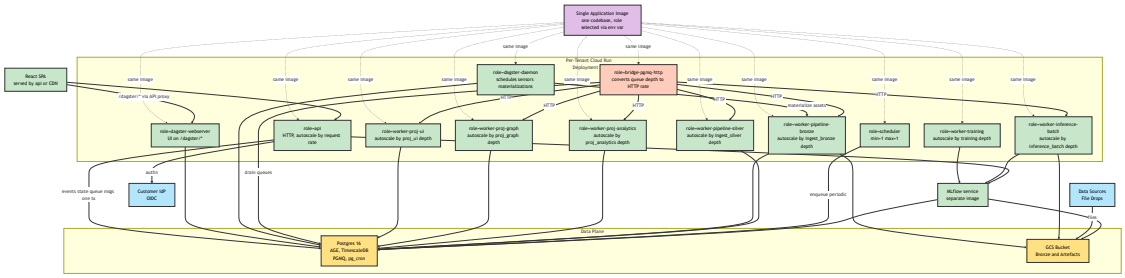

Figure 5.2: Container diagram

5.4.5 Identity and security

Concern	Component	Rationale
OIDC	**authlib**	Mature, standards-compliant, framework-agnostic
JWT	**PyJWT**	For session JWT minting and verification
Secret store (prod)	**Secret Manager**	Per-tenant IAM-scoped
Secret store (local)	`.env` via **python-dotenv**	Single line of environment-aware config

5.4.6 Infrastructure and delivery

Concern	Component	Rationale
Application runtime	**Cloud Run**	Stateless container, min-instance support, revision-based rollout
Job runtime	**Cloud Run Jobs** (CPU), **Vertex AI Custom Training** (GPU)	Single dispatch interface
Image registry	**Artifact Registry**	IAM-integrated, vulnerability scanning
Infrastructure as code	**Terraform**	Per-tenant modules; state in GCS
CI/CD	**GitHub Actions** + **Workload Identity Federation**	No long-lived keys
Container base	**Chainguard Python** or **Distroless**	Minimal attack surface, continuously patched upstream

5.4.7 Tooling

Concern	Component	Rationale
Package manager	**uv** (Astral)	Order-of-magnitude faster than pip/poetry on cold and warm installs (per Astral's published uv benchmarks); lockfiles, workspaces, Python version pinning; the current default for new Python projects in 2026
Linter/formatter	**ruff** (Astral)	Replaces flake8, black, isort, pyupgrade, etc. in one Rust binary
Type checker	**pyright** (or Astral's **ty** once GA)	Strict typing enforced in CI

Concern	Component	Rationale
Testing	**pytest**, **pytest-asyncio**, **hypothesis**	Unit + property-based
Integration testing	**testcontainers-python**, FastAPI **TestClient**	Real services in docker, real HTTP against the app
E2E testing	**Playwright**	Real browser, headless in CI
Observability	**OpenTelemetry** (Python SDK + Collector sidecar) -> **Cloud Logging** / **Cloud Monitoring** / **Cloud Trace**	Open standard, GCP-native sinks

5.4.8 Why these choices over common alternatives

- **Polars over pandas**: for the workloads typical of this platform — bulk silver/gold transformations, group-bys, joins on multi-million-row frames — Polars is materially faster than pandas and has substantially better memory characteristics. Its lazy API produces query plans that optimise across operations. Pandas remains acceptable at integration points with libraries that require it.
- **uv over pip/poetry/rye**: uv is orders of magnitude faster, handles Python version management, and has stable lockfile semantics. Container builds see minutes shaved off. Astral's continued stewardship and rapid release cadence secure its trajectory.
- **Nixtla suite over hand-rolled time-series pipelines**: the Nixtla libraries cover the full lifecycle (statistical, ML, neural, hierarchical) with a consistent API, are benchmarked against academic baselines, and are production-grade. Reinventing any part of this stack is no longer justified.
- **PGMQ over Redis/Kafka/Pub-Sub**: shares the transactional boundary with the state changes it announces; removes an entire class of dual-write inconsistency.
- **TanStack Query + Router over Redux / React Router**: TanStack's API is the preferred default for data-fetching and client routing in 2026 and is substantially smaller to learn and maintain.
- **FastAPI over Litestar**: Litestar's raw throughput is higher, but the ecosystem, documentation, and hiring pool for FastAPI are decisively larger, and the throughput gap is irrelevant at realistic request rates for this market.
- **Dagster over Airflow / Prefect, alongside APScheduler and PGMQ**: the three concerns coexist rather than compete. PGMQ carries the CQRS command-and-event flow because it shares the transactional boundary with state changes (no dual-write problem). APScheduler handles in-process cron, including the cron triggers that hand work off to Dagster. Dagster owns the data-and-ML asset graph: bronze, silver, and gold layers as software-defined assets, training runs and model versions as dynamic per-strategy assets, asset checks as first-class data-quality enforcement. Dagster's asset model is the visual lineage view an LP allocator wants to see during operational due diligence; that alone earned it a place in v1. Airflow's task-graph model is task-centric rather than asset-centric and a poorer fit for medallion data; Prefect is a credible alternative, but Dagster's asset checks and lineage UI are decisive for this market.

5.5 Data flow at a glance

A user action in the React UI issues a REST call to the API. The API validates the JWT session token, resolves the tenant context, and dispatches a command to the domain layer. The

command handler performs its validation, writes events to the event store, enqueues projection messages to PGMQ queues, and updates aggregate state — all within a single Postgres transaction.

Projection workers — running as separate Cloud Run services, one per projector role, each stamped from the same image with a distinct `ROLE` env var — consume their respective queues, transform events into read-model updates, and commit those updates to their target tables, graphs, or hypertables. Each worker implements internal concurrency via asyncio tasks, but does not co-reside with the API process. Subsequent user queries read from these projections.

In parallel, ingestion pipelines fetch data from external sources, land it in bronze (GCS), transform and validate it into silver (Postgres), and aggregate it into gold (Postgres). Training pipelines extract data from gold, run model training jobs (on Cloud Run Jobs or Vertex AI for GPU workloads), and register the resulting artefacts in MLflow. Serving endpoints load registered models and expose inference through the API.

Every step emits structured events. Every event is idempotent, versioned, and reversible by replay.

Chapter 6

Tenancy, Identity, and Authorisation

6.1 Silo tenancy model

Each customer receives a dedicated instance of the platform. There is no shared database, no shared application process, and no row-level isolation between customers. Tenant boundary is infrastructure boundary: a separate Cloud Run service, a separate Cloud SQL instance, a separate GCS bucket, all within a separate GCP project.

This choice makes the application code materially simpler — there is no tenant identifier on any row, no row-level security policies, no tenant context middleware, no noisy-neighbour reasoning — at the cost of higher per-tenant infrastructure floor. The trade is correct for this customer segment: hedge funds demand isolation, and the minimum infrastructure cost is well below their willingness to pay.

The control plane (described in the infrastructure chapter) maintains a registry of tenants and coordinates their lifecycle. The application itself knows only that it is running for one customer and reads its configuration from environment variables and Secret Manager on startup.

6.2 Identity federation

Customers authenticate their users through their own enterprise identity provider. Two provider types cover the target market:

- **Google Workspace** — an OIDC provider with Google's standard discovery endpoint, issuing ID tokens signed by Google
- **Microsoft Entra (formerly Azure AD)** — an OIDC provider with per-tenant issuer URLs, optionally enriched with group or application role claims

The application does not maintain user passwords, MFA enrolment, or session state beyond the scope of a single browser session. All authentication ceremony happens in the customer's directory. The application trusts the customer's identity provider to assert a verified user identity and builds its own session on top of that assertion.

6.3 Authentication flow

The flow follows the standard OIDC Authorisation Code exchange with PKCE. The application hosts four endpoints on its authentication router: `GET /auth/login`, `GET /auth/callback`, `POST /auth/refresh`, and `POST /auth/logout`.

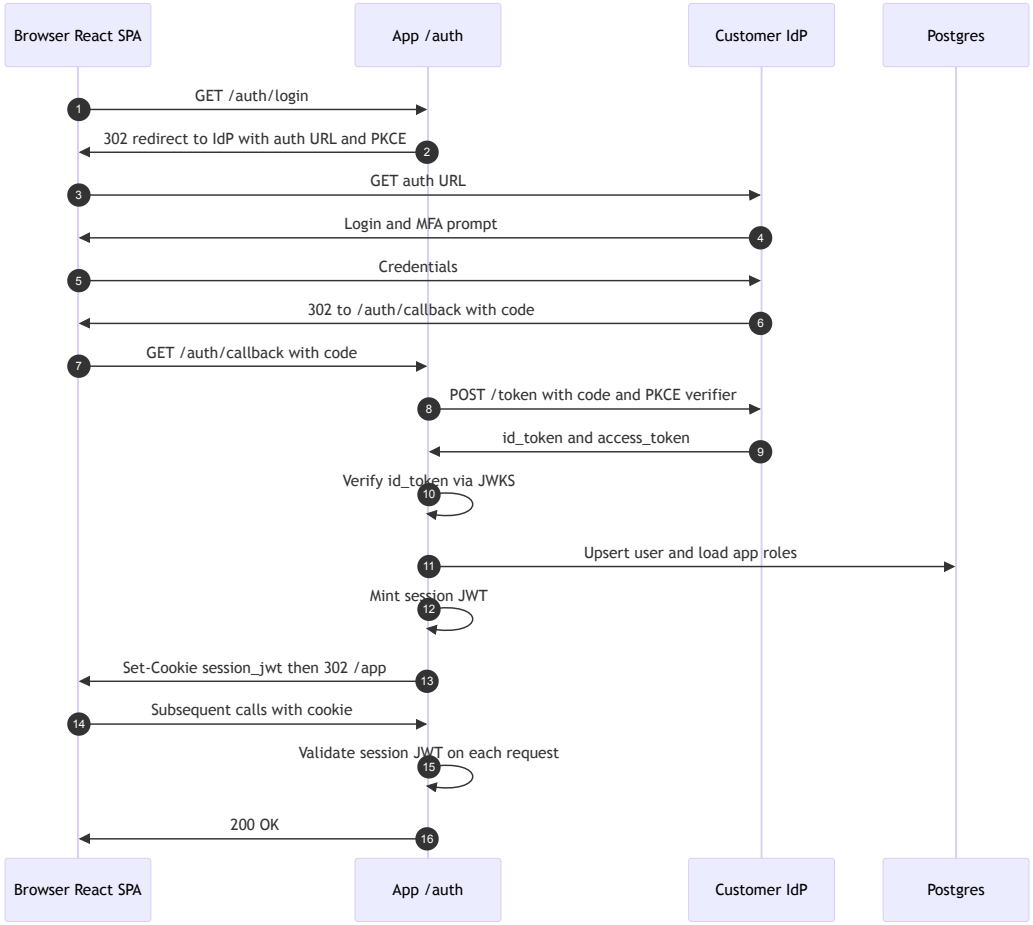

Figure 6.1: OIDC authentication sequence

On first request, the React SPA redirects to `/auth/login`, which constructs the authorisation URL against the customer's configured OIDC provider and redirects the browser. The user authenticates against their corporate directory. The provider redirects back to `/auth/callback` with an authorisation code. The application exchanges the code for an ID token, verifies the token's signature against the provider's JWKS, extracts the verified identity claims, and issues its own session JWT. The session JWT is returned to the browser as an HTTP-only, SameSite=strict cookie.

The session JWT is the only credential the React application ever handles. It contains the user identifier, the roles granted to the user in the application domain, and a short expiry (typically fifteen minutes). A longer-lived refresh token is stored separately and used by `/auth/refresh` to mint new session tokens without redirecting to the identity provider.

6.4 Session JWT content

The session token is signed by the application using a key stored in Secret Manager (rotated on a schedule). It carries only the claims the application itself needs:

- **`sub`** — a stable internal user identifier, derived from the external provider's subject claim
- **`email`** — the user's email address, for logging and display
- **`roles`** — application-specific role strings (e.g. `quant`, `risk`, `admin`, `viewer`)
- **`iat`, `exp`** — issuance and expiry timestamps
- **`jti`** — a unique token identifier, used for revocation

The token is validated on every request by a FastAPI dependency that also populates `request.state.user` with the parsed claims. No route handler reads cookies directly or parses tokens; authentication is transparent to business logic.

6.5 Role mapping

Customers' identity providers do not know about the application's role vocabulary. They may emit group memberships (Workspace) or application roles (Entra), but the semantics are the customer's, not the application's.

A `user_roles` table in Postgres maps authenticated users to the application's role vocabulary:

Column	Purpose
`user_sub`	Stable external subject from the identity provider
`email`	Captured at first login for operational convenience
`roles`	Array of application role strings
`created_at`, `updated_at`	Audit fields

Two patterns populate this table:

1. **Administrative assignment** — a platform administrator invites users by email and assigns roles before first login. On first authenticated request, the application matches the email claim against the pending invitation and activates the row.
2. **Claim-based mapping** — for customers who have configured their identity provider to emit group or role claims (Entra application roles are the cleanest path), a configuration table maps external claim values to internal roles. On each login, the mapping is applied, and the `user_roles` row is refreshed.

The application never trusts the external provider's groups directly. External claims are inputs to a mapping function owned by the application; roles attached to the session JWT are the output.

6.6 Authorisation enforcement

Authorisation is enforced in two places:

- **At the route** — a FastAPI dependency decorates each protected endpoint with the roles it requires. Requests from users lacking the required role receive HTTP 403.
- **At the domain** — within command handlers, fine-grained checks apply to entity-level permissions (e.g. a `quant` user may edit their own models but not another user's).

The route-level check is coarse and mandatory; the domain-level check is fine and context-aware. Both are unit-tested.

6.7 Multi-provider support per instance

Each instance is configured with exactly one identity provider. A customer organisation running multiple directories (e.g. a parent company with subsidiaries on different tenants) is served by multiple silo instances, each bound to its own directory. There is no need for intra-instance multi-provider routing, because there is no pool tenancy.

This keeps the auth code path trivially simple: one issuer, one JWKS URL, one client ID, one client secret, one group-mapping configuration. All of these are loaded from Secret Manager at startup.

6.8 Service-to-service authentication

Internal components (the application calling MLflow, the application invoking Cloud Run Jobs for training, the CI/CD pipeline deploying revisions) authenticate using GCP service accounts and Workload Identity Federation. No static credentials, API keys, or service account JSON files exist in the codebase or the container images.

The application's service account is granted the minimum IAM roles required to access its Cloud SQL instance, its GCS bucket, its Secret Manager secrets, and its MLflow backend. Cross-tenant access is prevented at the IAM boundary, not at the application layer.

Chapter 7

Application Architecture

7.1 Single image, multiple roles

The application is built, shipped, and operated as a single Docker image. That image, when instantiated, picks a **role** from an environment variable at startup and behaves accordingly. Roles supported in a typical deployment:

Role	Responsibility	Deployed as
`api`	HTTP request/response, command dispatch, query serving, OIDC callbacks	Cloud Run service, autoscaled by HTTP request rate
`worker-proj-ui`	Projects events into the UI read-model tables	Worker service, autoscaled by `proj_ui` queue depth
`worker-proj-graph`	Projects events into the AGE graph	Worker service, autoscaled by `proj_graph` queue depth
`worker-proj-analytics`	Projects events into TimescaleDB aggregate tables	Worker service, autoscaled by `proj_analytics` queue depth
`worker-pipeline-bronze`	Handles bronze-arrival events, triggers silver transforms	Worker service, autoscaled by `ingest_bronze` queue depth
`worker-pipeline-silver`	Silver-to-gold transformation	Worker service, autoscaled by `ingest_silver` queue depth
`worker-training`	Training job dispatch and status polling	Worker service, autoscaled by `training` queue depth
`worker-inference-batch`	Batch inference job processor	Worker service, autoscaled by `inference_batch` queue depth
`scheduler`	APScheduler daemon emitting periodic messages	Single-instance service, min=1 max=1

Every role is the same image. The same test suite. The same build. The same deployment pipeline. The difference is the `ROLE` environment variable.

The application's `main` module reads the role on startup, sets up the appropriate components (FastAPI ASGI app for `api`; asyncio worker loops for `worker-*` roles; APScheduler for `sched-`

uler), and begins its loop. Because the image is identical, a developer can run any role locally with a simple environment variable change and can be confident that what runs locally matches what runs in production.

7.2 Rationale

The two alternatives commonly proposed for this problem space are explicitly rejected.

Scattered serverless — one Cloud Function or Cloud Run service per task, each with its own repository, its own CI/CD pipeline, its own dependency list. Fragments the codebase, makes local development heavy, multiplies operational surface, and typically introduces duplicate implementations of cross-cutting concerns (logging, auth, database access). The worst version of this pattern is a team maintaining forty cloud functions across five repositories, none of which are individually testable end to end.

Single-process kitchen sink — one `uvicorn` process hosting the API plus every worker as an asyncio task. Simpler to set up; fatal in production. Workers cannot scale independently of the API; backpressure on one worker starves the rest; queue-depth autoscaling has no lever to pull because the worker capacity is pinned to whatever the API container can support. Adequate for a prototype, inadequate for a production platform.

The single-image-multi-role pattern is the middle path that mature distributed systems converge on. It preserves the codebase simplicity of a monolith (one test suite, one build, one deployment) while providing the runtime independence of microservices (one scaling policy per workload, one failure domain per role, one observability scope per concern).

7.3 Role selection and startup

The application's entry point is `python -m app.main`. The `main` module:

1. Reads the `ROLE` environment variable (default: `api`).
2. Loads configuration (database connection, Secret Manager bindings, MLflow URL) — common to all roles.
3. Performs health-check bootstrap (connects to Postgres, verifies extensions, confirms MinIO/GCS access) — common to all roles.
4. Branches on role:
   - `api` -> starts `uvicorn` hosting the FastAPI app
   - `worker-*` -> starts the specified worker loop as an asyncio task, plus a minimal HTTP health-check endpoint for Cloud Run or GKE probes
   - `scheduler` -> starts APScheduler with the job registry plus a health-check endpoint

Each branch shares the application-level observability setup: structured logging with a `role` label, OpenTelemetry tracing, Cloud Monitoring metrics export. A log line from any role is instantly identifiable by the `role` field.

7.4 Command-Query Responsibility Segregation

The write and read paths are separated. Commands are handled by the `api` role in the main request-response path; read models are maintained by the `worker-proj-*` roles consuming events from PGMQ queues.

7.4.1 Command path

A user action flows as:

1. The REST handler (running in the `api` role) receives the request, validates the input via Pydantic schemas, and constructs a command object.
2. A command dispatcher loads the relevant aggregate from the event store (by replaying events or by loading a snapshot).
3. The aggregate applies domain invariants and produces a sequence of events.
4. Within a single Postgres transaction:
   - The events are appended to the `events` table.
   - The aggregate's state snapshot is updated.
   - Messages are enqueued to each PGMQ queue that serves a read model dependent on these events.
5. The transaction commits atomically, or the entire operation fails.

The transactional outbox pattern is trivialised because PGMQ lives in the same database as the event store. There is no separate broker to synchronise with, no outbox relay process, no dual-write problem.

7.4.2 Projection path

Each read model is served by a dedicated worker role (`worker-proj-ui`, `worker-proj-graph`, `worker-proj-analytics`). That worker runs an asyncio loop that polls its PGMQ queue with a visibility timeout, processes messages in parallel up to a configured concurrency, updates its target read model, and acknowledges.

Projector workers are horizontally scaled. Multiple replicas of `worker-proj-graph` can consume the same queue concurrently; PGMQ's visibility timeout prevents double-processing. When the `proj_graph` queue depth grows, the autoscaler adds replicas; when it drains, replicas scale down.

Projectors are idempotent. Each maintains a `processed_events` table keyed by `(projector_name, event_id)` with a uniqueness constraint. Reprocessing is safe; rebuilding a projection from scratch is truncating its state tables and resetting its watermark.

7.5 Worker anatomy

Every worker role follows the same structure:

- **Startup** — load configuration, connect to Postgres, register the worker's queue name(s) with the PGMQ client.
- **Main loop** — `pgmq.read(queue, vt=N, qty=K)` to pull a batch; process the batch with bounded concurrency using `asyncio.gather` plus a semaphore; on success, `pgmq.delete(queue, msg_id)`; on failure, let the visibility timeout expire and the message will be redelivered.
- **Retry and DLQ** — after the configured retry count, the message is archived to a dead-letter table with the failure reason.
- **Graceful shutdown** — on SIGTERM, stop pulling new messages, finish in-flight processing, then exit. Cloud Run and GKE both respect the termination grace period.
- **Health checks** — a minimal HTTP endpoint (usually port 8080) returns 200 when the worker is healthy, 503 when unable to reach Postgres or stuck. Cloud Run and GKE use this for liveness and readiness probes.

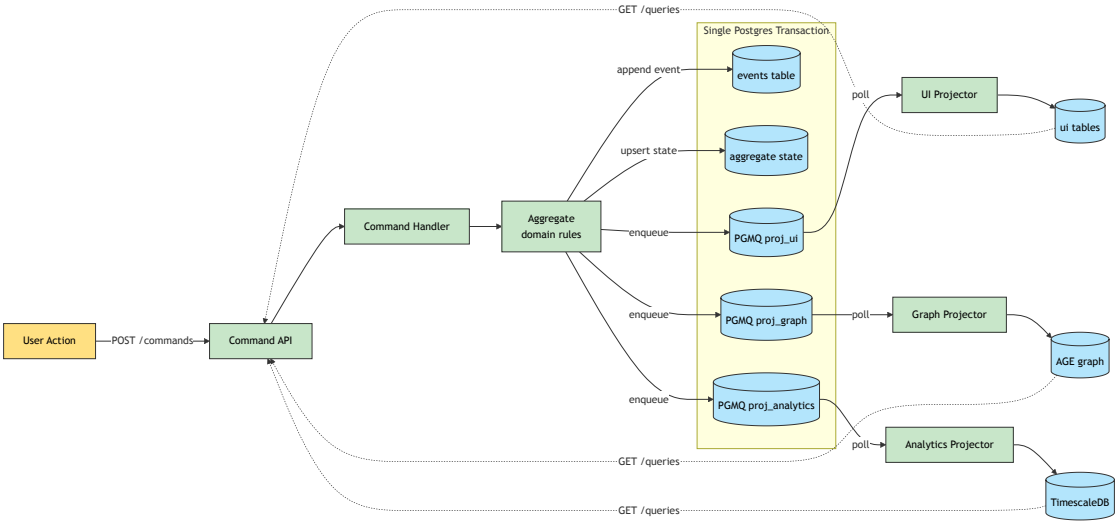

Figure 7.1: CQRS command and projection flow

The base Worker class implements all of this. Specific worker roles subclass it and provide only a `handle(message)` coroutine; the registry file maps role names to handlers.

7.6 REST API structure

The API surface is organised into routers by concern:

Router	Path prefix	Purpose
Authentication	`/auth`	OIDC login, callback, token refresh, logout
Commands	`/commands`	State-mutating actions
Queries	`/queries`	Read access to projections
Training	`/training`	Model training submission and status
Serving	`/serving`	Model inference endpoints
Admin	`/admin`	User management, role assignment, tenant configuration
Internal	`/internal`	Health checks, worker status, PGMQ dead-letter inspection

Every route carries an OpenAPI specification generated by FastAPI. The React client is generated from the OpenAPI spec using `openapi-typescript`, keeping the client-server contract in lockstep.

7.7 Domain model

The domain layer contains no framework or infrastructure dependencies. Aggregates are Python classes with pure methods. Events and commands are Pydantic models. Domain tests exercise business rules with plain function calls — no database, no HTTP, no workers.

Typical aggregates:

- **Model** — a trained quantitative model with a version history, lifecycle state (draft, validated, promoted, retired), and owning team.
- **Dataset** — a versioned snapshot of input data, with lineage to bronze sources.
- **TrainingRun** — a single execution of a training pipeline with inputs, outputs, metrics, and reproducibility metadata.
- **Scenario** — a what-if analysis request carrying a model version, input dataset, and result set.
- **RiskLimit** — a threshold (VaR, position, drawdown) that scenarios and live inference checks are validated against.

Aggregates emit events that are versioned, immutable, and replayable.

7.8 React frontend

The frontend is a single-page application built with Vite, consuming the REST API exclusively. It is deployed as static assets — HTML, JavaScript, CSS — produced by the frontend build.

Two hosting options are supported by the same build artefact:

1. **Served by the `api` role** — static files bundled into the container, served by a FastAPI static-files middleware at the root path. Simpler operations, a single deployable, adequate for customer-scale traffic.
2. **Served by Cloud CDN** — static files uploaded to a GCS bucket, fronted by Cloud Load Balancing with Cloud CDN. Lower first-byte latency, cacheable edge delivery. Preferred for larger customer bases.

Either way, authentication cookies are scoped to the application's domain, and API calls are same-origin. There is no separate frontend domain, no CORS configuration, and no cross-site cookie handling.

7.9 Testability

Every layer has a clear test seam:

- **Unit tests** exercise the domain layer with no I/O. Hundreds of tests running in seconds.
- **Integration tests** exercise the API, persistence, and projection layers against the docker-compose stack — real Postgres, real PGMQ, real MinIO. Every production code path is tested here.
- **Contract tests** validate that old events still deserialise after schema migrations and that the OpenAPI specification matches the generated frontend client.
- **End-to-end tests** drive the React UI against a spun-up local stack using Playwright.

No test relies on cloud access. No test uses mocks where a real component is available locally.

7.10 Local development and role execution

Locally, each role can be run in its own container (true production parity) or, for convenience during the fastest inner development loop, multiple roles can run in one process with asyncio tasks. Both modes are supported by the same image. The `make dev` target brings up the docker-compose stack and starts the `api` role plus one replica of each worker role; the `make dev-fast` target runs all roles in a single process for engineers who want faster restart cycles at the cost of production parity. The local development chapter describes this in detail.

Chapter 8

Data Platform

8.1 Medallion architecture

The data platform follows the medallion pattern: incoming data lands in a raw **bronze** layer, is cleaned and typed into a **silver** layer, and is aggregated into a **gold** layer that serves the application's read models, the training pipelines, and downstream analytics.

8.1.1 Bronze

Raw data as received, stored in object storage (GCS in production, MinIO locally) as immutable parquet files. The storage path encodes tenant (implicit — the bucket is per-tenant), source system, ingestion date, and a content hash:

`gs://{tenant-bucket}/bronze/{source}/{yyyy}/{mm}/{dd}/{file_hash}.parquet`

Files are never overwritten. A re-delivery from the source produces a new file with a new hash. File lifecycle is managed by bucket lifecycle rules — Nearline at thirty days, Coldline at ninety days, deletion at the configured regulatory retention horizon (typically seven years for hedge-fund workloads).

A `bronze_files` table in Postgres records every arrival: tenant, source, URI, hash, received time, size, and processing status. This table is the authoritative inventory of ingested data and is the basis for incremental processing.

Accepted on-the-wire formats include CSV, JSON, and parquet. Non-parquet inputs are converted to parquet on arrival and the original format is preserved alongside for replay.

8.1.2 Silver

Cleaned, typed, and validated data held in Postgres tables, partitioned or hypertable-organised for time-series sources. Silver is the operational source of truth for all downstream consumption.

Each silver table carries lineage columns — `_bronze_uri`, `_bronze_hash`, `_ingested_at` — linking every row back to the raw file it came from. Data quality validation is performed in-flight: a row that fails validation is routed to a `quarantine` table with a reason code, never silently dropped.

Silver transformations are implemented as Python functions using Polars (preferred) or pandas (for legacy libraries). Each function is independently testable, idempotent (upsert by natural

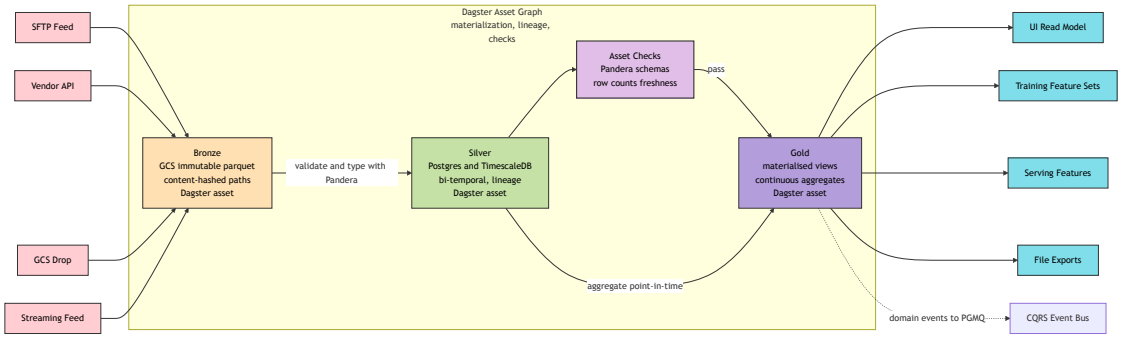

Figure 8.1: Medallion data flow

key), and exposed as a Dagster software-defined asset whose materialization is triggered either by APScheduler on a cron cadence or by a PGMQ message emitted on bronze arrival.

A `ingestion_watermark` table records per-source processing progress — `(source, last_processed_hash, last_processed_at)` — to support incremental loads without reprocessing.

8.1.3 Gold

Business-level aggregates served by Postgres tables, materialised views, or TimescaleDB continuous aggregates. Gold is shaped by consumer: a read model for the UI, a feature set for model training, an analytical view for internal reporting.

Gold tables emit domain events on update. The event flows to the CQRS event store and to any projection subscribed to gold-layer changes. This closes the loop between the data platform and the application: pipeline outputs become first-class events that drive read models and trigger downstream actions (retraining, alerting, compliance checks).

8.2 File-based ingestion and output

The realistic operating mode for hedge-fund customers is file-based. Data vendors deliver files. Prime brokers send files. Custodians publish files. Fund administrators receive files. The platform accordingly treats file exchange as the primary integration surface, with streaming treated as a specialised case rather than the default.

8.2.1 Inbound ingestion patterns

The platform supports four inbound patterns, each covering a different category of source:

Scheduled pull — the platform reaches out to the source on a cron cadence and retrieves new files. Applicable to SFTP servers, HTTPS endpoints returning file listings, and vendor APIs that support “list since timestamp” semantics. APScheduler triggers a pull job that authenticates (using credentials from Secret Manager), enumerates new files, downloads them, computes their content hashes, and writes them to the bronze bucket. Idempotency is enforced by content hash: a file whose hash is already in the `bronze_files` registry is skipped.

Push drop to GCS — the source uploads directly into the tenant’s GCS bucket. This is the cleanest mechanism when the source supports it (most modern data vendors do). A GCS object-notification trigger emits an event that is received by a small webhook endpoint on the application, which enqueues a bronze-processing message to PGMQ. No polling, no scheduler, minimum latency from arrival to processing.

Push drop via SFTP to vendor landing — for legacy sources that cannot reach GCS but can reach an SFTP server, the vendor writes to a shared SFTP landing operated by the vendor’s own infrastructure. A scheduled pull job polls the SFTP landing and forwards new files into the customer’s bronze bucket. This degrades gracefully to “scheduled pull” when direct GCS push is infeasible.

Customer-operated drop — the customer’s own systems (trade booking, risk, compliance) write files directly into the tenant’s bucket on their side of the integration. This is the pattern when the customer is sourcing their own data for their own models. The customer retains control of the production schedule and the delivery contract; the platform consumes what arrives.

Streaming ingestion is provided for sources that genuinely emit messages rather than files (market data multicast feeds, FIX execution messages). The streaming path is architecturally a special case of the file path: a persistent consumer micro-batches incoming messages into parquet files on short tumbling windows and writes them to bronze, after which the same silver-transformation code paths apply. This allows the rest of the platform to remain file-oriented regardless of the raw source shape.

8.2.2 File contracts

Every source carries a versioned file contract that specifies:

- Expected file name pattern (with date tokens and sequence numbers)
- Expected format (CSV, parquet, or JSON; for CSV, delimiter, encoding, and header-presence options)
- Expected schema (column names, types, nullability, value domains)
- Expected delivery cadence (daily by 08:00 UTC, intraday every fifteen minutes, etc.) and tolerance windows
- Optional manifest file (a companion file listing expected content hashes, row counts, or checksums, allowing integrity verification before processing)
- Late-arrival policy (how far back corrections are accepted; how late deliveries are handled)

The file contract is machine-readable. The ingestion pipeline reads it, validates arrivals against it, and quarantines violations rather than attempting to muddle through. A CSV with an unexpected column is not silently mis-mapped; it is rejected with a specific reason code and surfaced to operations.

8.2.3 Outbound delivery patterns

Model outputs, scheduled inference results, and exports flow out of the platform through an inverted set of the same patterns:

Scheduled file export to customer GCS bucket — the platform writes outputs to a customer-specified bucket on the customer's schedule. IAM on the customer's bucket grants the platform service account write access; no credentials are exchanged. The customer's downstream systems poll their own bucket and consume.

Scheduled SFTP push — for customers whose downstream systems cannot read from GCS, the platform uploads outputs to a customer-provided SFTP endpoint using credentials in Secret Manager. This path is explicitly supported, but discouraged; GCS-native delivery is preferred.

On-demand signed URL download — for ad-hoc or user-initiated exports, the platform writes the result to a short-lived location in its own bucket and returns a signed URL valid for a configured window (typically fifteen minutes to one hour). The user downloads directly from GCS; the platform never streams large payloads through its own API.

API-returned batch results — for small outputs (single-file inference results, score sheets, report summaries), the API returns the file content directly in the response. This path is available but bounded by a maximum response size; larger outputs are forced through signed URLs.

Every outbound delivery is recorded in an `outbound_deliveries` table with the target, the file URI, the content hash, the scheduled time, the actual delivery time, and the success status. The table is the audit record of what was sent to whom, queryable by the customer and by internal operations.

8.2.4 Reconciliation and recovery

File-based integration produces a characteristic class of failure: the file never arrives, the file arrives malformed, the file arrives on time but with stale data, the file is duplicated. The platform handles each:

- **Expected-delivery monitoring** — the scheduler knows when each source is expected to deliver. If the tolerance window passes without arrival, an alert fires.
- **Content-hash deduplication** — the `bronze_files` registry prevents a re-delivered file from re-entering silver; the row count tells operations that a duplicate was detected.
- **Manifest verification** — where manifests are provided, mismatch triggers quarantine and an operations alert before downstream processing.
- **Row-count reconciliation** — silver-to-gold loads emit row-count deltas; anomalies (a drop of 90% from yesterday) page operations before contaminated data reaches model training or serving.
- **Backfill and replay** — the `bronze_files` registry supports re-emission of bronze-ingested messages for a specified date range, causing the full silver-and-gold pipeline to re-execute. This is the recovery mechanism when a source publishes corrections for past dates.

8.3 Transformation and validation

Transformations are plain Python functions organised under `app/pipeline/`, grouped by layer (`bronze_to_silver`, `silver_to_gold`) and by domain (`positions`, `prices`, `trades`, `reference_data`). Each function has a single responsibility, a typed input and output, and a matching unit test.

Validation uses Pandera (preferred for Polars/pandas DataFrames) or pydantic (for row-level validation of smaller datasets). Every silver-layer load applies the source's validation schema; failures are routed to `quarantine` rather than failing the pipeline outright.

Data quality metrics — row counts, null rates, distribution shifts — are emitted as observability signals at the end of every pipeline run. Thresholds are per-source and per-tenant; breaches trigger alerts without blocking the pipeline (except for critical sources where block-on-violation is explicitly configured).

8.4 Scheduling and orchestration

Three components share the orchestration surface, each handling a different concern:

- **PGMQ** carries the CQRS command-and-event flow. It lives in the same Postgres instance as the aggregate state it announces, so a command handler can update state and enqueue projection messages in one transaction. This is the substrate for the in-application event loop, not for the pipeline graph.
- **APScheduler** is the in-process cron. It triggers Dagster materialization runs on a cadence and handles the small set of recurring jobs (token refreshes, watermark sweeps, expected-delivery monitors) that do not warrant their own asset.
- **Dagster** is the data-and-ML pipeline orchestrator. Bronze, silver, and gold layers are modelled as software-defined assets; training runs and model versions are dynamic assets per strategy. Asset checks make data-quality enforcement first-class. Dagster's asset graph is the lineage view that quants, operations, and LP-allocator due-diligence reviewers all want to see.

The asset model fits the medallion architecture cleanly. Each silver transformation is an asset whose upstream is the bronze file (or files) it consumes; each gold aggregate is an asset whose upstream is the silver tables it reads. A row arriving in bronze does not have to be matched against a hand-maintained dependency table — the dependency *is* the asset graph, and Dagster materialises downstream assets in the correct order with the correct partitions. Asset checks attach validations (row counts, null rates, distribution tests) directly to the asset, so a data-quality breach surfaces in the same place as the asset's lineage and run history.

The end-to-end flow for a typical bronze-to-gold path:

1. A bronze file arrives (via scheduled pull or push drop) and a `bronze_ingested` message lands on PGMQ.
2. A small worker consumes the message and triggers materialization of the corresponding silver asset in Dagster (for event-driven sources) or APScheduler triggers a periodic materialization of a window of silver assets (for cron-driven sources).
3. Dagster materialises the silver asset, runs its asset checks, and propagates the materialization to downstream gold assets according to the asset graph.
4. Gold materialization emits a `gold_updated` domain event back onto PGMQ, which projectors, training triggers, and alerting rules subscribe to.

The Dagster UI is exposed read-only to operators and to quants through the BFF on the `/dagster/*` proxy path. The same UI surfaces the visual DAG, partition status, asset-check results, and run history. For an LP allocator running operational due diligence, the lineage view is concrete evidence that gold-layer aggregates trace cleanly back to bronze files with no off-graph manual steps.

Run storage uses the existing Postgres instance — Dagster does not introduce a new database. In local development Dagster is a docker-compose service on port 3000; in production it is a Cloud Run service in the tenant project. Backfills are first-class Dagster operations, invokable from the UI or from a CLI; the previously CLI-only backfill path is retained for headless and scripted use.

The argument for Dagster as a v1 component (rather than the deferred component it was in earlier drafts) is the lineage UI and the asset-check model. APScheduler plus PGMQ alone can drive the pipeline; what they cannot do is *show* the pipeline. The asset graph is the visual artefact a non-engineering reviewer can read, and the asset checks are the structural place to encode data-quality rules. Both are shippable as part of the v1 demo without new infrastructure.

Airflow and Prefect were considered. Airflow's task-graph model is task-centric rather than asset-centric and a poorer fit for medallion data; Prefect is a credible alternative, but Dagster's asset checks and lineage UI are decisive for this market segment.

8.5 Point-in-time correctness

Quantitative workloads demand point-in-time correctness — a model training run must see the data as it was known on the training date, not as it has been subsequently corrected. The platform enforces this with a bi-temporal schema that captures **both** halves of the bi-temporal model on every silver and gold row:

- `_knowable_at` — system time. The timestamp at which the datum first became visible to the platform (typically the bronze-landing time of the file that carried it). This is the column training-extraction queries filter on.

- `_valid_from` / `_valid_to` — valid time. The business-time interval over which the datum applies. Corrections replace the interval rather than overwrite the row.
- The original business-event timestamp on the underlying fact remains in its own column (e.g. `trade_date`, `observation_date`), untouched by the bi-temporal columns above.

Training pipelines always specify an `as_of` timestamp and filter `_knowable_at <= :as_of`. Extraction queries that lack this filter fail pipeline validation before they can ship. Bi-temporal reconstructions ("what did we believe on date T about observations over period P?") combine the two halves: filter `_knowable_at <= T` and intersect with the valid-time interval covering P.

Corrections to historical data create new rows rather than updating old ones. The old row's `_valid_to` is set to the correction's effective time; the new row's `_valid_from` matches, and its `_knowable_at` reflects when the correction itself became visible. This allows the platform to answer both "what did we know at time T?" (system time, for training reproducibility) and "what was true for period P as currently understood?" (valid time, for reporting).

TimescaleDB's support for this pattern via hypertable constraints and continuous aggregates on time-bounded ranges is one of the reasons for its inclusion in the default stack.

Chapter 9

Model Training and Serving

9.1 Separation of training and serving

The platform cleanly separates model training — a compute-heavy, often long-running, sometimes GPU-dependent activity — from model serving — a latency-sensitive, always-on, request-response activity. Both are first-class capabilities of the platform, but they run on different infrastructure with different scaling characteristics and different deployment cadences.

9.2 Training environment

9.2.1 Workload characteristics

Training jobs in a quantitative context fall into three categories, each with different infrastructure needs:

1. **Feature engineering and small-model training** — minutes to hours, CPU-only, memory-bound. Examples: linear factor models, tree-based risk scoring, rolling-window statistics. Runs comfortably on Cloud Run Jobs with 4–16 vCPU and 16–64 GB of memory.
2. **Medium deep learning** — hours to a day, single GPU, occasionally multi-GPU. Examples: LSTM price predictors, transformer-based signal generators, autoencoders for anomaly detection. Requires GPU-enabled compute: either Cloud Run with GPU (limited regions), GKE Autopilot with a GPU node, or Vertex AI Custom Training.
3. **Large-scale training and hyperparameter search** — days, multi-GPU, distributed. Uncommon in traditional quant but relevant for firms exploring foundation-model approaches. Requires Vertex AI Training with multi-worker configurations or GKE with dedicated GPU pools.

The platform dispatches to the appropriate compute target based on job metadata. Small jobs stay inside the main application's Cloud Run service (a synchronous or short async training). Medium and large jobs are submitted as Cloud Run Jobs, GKE jobs, or Vertex AI Custom Training jobs and tracked asynchronously.

9.2.2 Training orchestration

A training run is a first-class aggregate in the domain model. It carries:

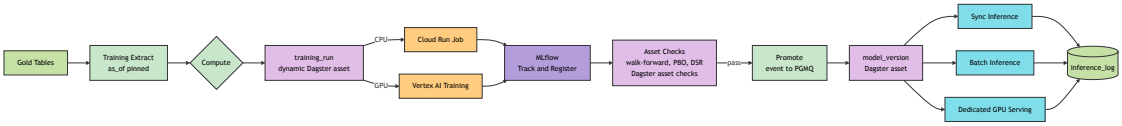

Figure 9.1: ML training and serving pipeline

- The model identity and version being trained
- The input dataset version (a pinned snapshot of gold-layer data)
- The training configuration (hyperparameters, loss function, validation split)
- The target compute environment
- The lifecycle state (submitted, running, evaluating, completed, failed)
- Metrics (loss curves, validation scores, backtest results)
- Artefact references (model weights, evaluation reports, serialised pipelines)

Training is triggered through the `/training/runs` API endpoint. The handler records a `TrainingRunSubmitted` event, dispatches the compute job to the target environment, and returns a run identifier. A background worker polls the compute environment for status and emits events on lifecycle changes.

The training pipeline itself is orchestrated by Dagster (introduced for the data layer in the previous chapter and reused here). A `training_run` is a dynamic Dagster asset per strategy, with the gold-layer assets it consumes as upstream dependencies; a `model_version` is the downstream asset that materialises when training completes, validation gates pass, and the artefact is registered in MLflow. The asset-check model carries the platform's validation gates — point-in-time correctness checks, walk-forward thresholds, calibration tests — so a model that fails validation cannot become a materialised `model_version` asset. The same lineage view that traces gold aggregates back to bronze files extends one more hop to show which `training_run` consumed which gold snapshot to produce which `model_version`. The SDK's `register()` call translates a strategy specification into the corresponding Dagster asset definitions; researchers do not write Dagster code by hand.

The actual training code runs in the same container image as the main application. A dedicated training entrypoint selects the training module by job type, loads the pinned dataset, executes the training loop, logs metrics and artefacts to MLflow, and exits. This means the training environment is testable locally: the same entrypoint runs in docker-compose with a small dataset and produces a real MLflow run.

9.2.3 Data extraction and reproducibility

Training data is extracted from gold-layer tables with an explicit `as_of` timestamp. The extraction query, its result hash, and the `as_of` are all recorded in the MLflow run. A training run can be reproduced at any later date by re-extracting with the same `as_of`, the same query, and confirming the hash matches.

Bi-temporal correctness (see the data platform chapter) guarantees that corrections to historical data after the original training date do not contaminate reproductions.

9.2.4 Point-in-time correctness and look-ahead bias

The quantitative-finance literature consistently identifies look-ahead bias — the inadvertent use of information that would not have been available at the decision time being simulated — as a leading cause of quantitative strategies appearing profitable in backtests and failing in production (López de Prado, *Advances in Financial Machine Learning*, Wiley 2018, Ch. 11–13; Bailey & López de Prado, "The Probability of Backtest Overfitting," *Journal of Computational Finance*, 2014). A representative example: a consumer credit-card transaction occurs on Monday, is aggregated by the data vendor on Wednesday, and arrives in the customer's data lake on Thursday. A naive backtest attributes the transaction to Monday and trades off it. A correct backtest attributes it to Thursday, when it was actually knowable, and obtains materially different results.

The platform enforces point-in-time correctness as a pipeline property, not as a researcher's discretionary practice. Every silver and gold table carries a `_knowable_at` column (system time — when the row became visible to the platform) alongside `_valid_from` / `_valid_to` columns (the business-time interval over which the datum applies); both are distinct from the original business-event timestamp. Every training extraction query filters by `_knowable_at <= :as_of`; queries lacking this filter fail validation at pipeline build time and cannot ship to production. The Data Platform chapter documents the column scheme in full.

9.2.5 Walk-forward validation

A single train-test split against historical data is insufficient evidence of a model's production viability. The platform's validation harness implements walk-forward validation as a first-class operation: the training window is advanced through historical time in steps, the model is re-trained at each step with strictly pre-step data, and performance is measured on the next step's out-of-sample period. The result is a sequence of out-of-sample performance observations rather than a single retrospective number, and it is dramatically more predictive of live-trading outcomes.

Walk-forward validation is a gating criterion on model promotion. A model whose walk-forward performance fails the configured threshold cannot transition from staging to production in the MLflow registry, regardless of the developer's preferences. This is a platform rule, not a team convention.

9.2.6 Research-to-production code parity

The second structural cause of backtest-to-production failure is code divergence between the researcher's notebook and the production pipeline: a feature computed one way in research and a different way in production. The platform collapses this divergence by requiring that the feature-extraction code used at training time is the same callable used at serving time, packaged into the model's `pyfunc` wrapper and invoked identically in both contexts. Research notebooks import the same wrapper module when prototyping, making the notebook the draft of the production serving path rather than a parallel artefact.

9.3 MLflow as the model registry and tracking backbone

The platform uses MLflow as the experiment tracking, model registry, and artefact storage coordinator. MLflow is open source, runs fully locally in docker-compose, and is backed by Postgres and GCS — both components already in the stack.

The MLflow tracking server is deployed as a separate Cloud Run service per tenant. It shares the Postgres instance with the main application (distinct schema) and uses the tenant's GCS bucket for artefact storage. Its upgrade cadence is decoupled from the main application for MLflow application versions — MLflow releases move slowly and are low-risk, whereas the application ships features continuously. The decoupling does not extend to the Postgres engine version itself: a Postgres major-version bump or an extension change requires coordination across both consumers and is managed at the tenant-upgrade level. Such changes are rare.

9.3.1 Experiments, runs, and models

MLflow's concepts map cleanly onto the platform's vocabulary:

- **Experiment** — a named grouping of related training runs for a given model family

- **Run** — a single execution of a training pipeline, containing logged parameters, metrics, and artefacts
- **Registered model** — a named model with a version history
- **Model alias** — a mutable pointer (e.g. `staging`, `production`, `champion`, `challenger`) attached to a specific version of a registered model

The platform's domain uses the Model aggregate as the user-facing entity and maps its lifecycle operations onto MLflow registry calls. A user clicking "promote to production" on a model version triggers a domain command that moves the `production` alias to that version and emits a `ModelPromoted` event. The platform pins MLflow at version 2.16 or later (where Model Aliases are first-class and Model Stages are deprecated) and follows the alias-based lifecycle rather than the deprecated stage transitions; migration to MLflow 3.x, which removes stages entirely, is a non-disruptive engine upgrade because the platform never relied on stages.

9.3.2 Model packaging

Models are packaged using MLflow's `pyfunc` flavour, which captures both the serialised model and its inference wrapper code. The wrapper is responsible for feature transformations, input validation, and output shaping. This means the serving code is part of the model artefact, not part of the serving infrastructure. Upgrading a model cannot break the serving endpoint's calling contract.

9.4 Serving

9.4.1 Inference modes

The platform supports three serving modes, all exposed through the application's REST API:

- **Synchronous inference** — a single request carrying a feature vector or identifier, returning a prediction within a few hundred milliseconds. Endpoint: `POST /serving/{model-name}/predict`.
- **Batch inference** — a request referencing a dataset (by ID) or a file (by GCS URI), returning a job identifier. Results are written to a caller-specified GCS location or made available through a subsequent query endpoint. Suitable for end-of-day scoring, portfolio-wide risk calculation, or backtest sweeps.
- **Scheduled inference** — a recurring pipeline configured at the model level, producing outputs on a cadence (e.g. daily post-close predictions). Scheduled runs are tracked in the `pipeline_runs` table alongside data pipelines.

9.4.2 Serving architecture

Serving endpoints are handlers within the main application process. On startup, the serving module reads a registry of models carrying the `production` alias from MLflow, downloads each model to a local cache directory, and holds references in memory. Requests are dispatched to the appropriate model by name and version.

This avoids the complexity of a separate model-serving service (Triton, BentoML, Seldon) at the cost of coupling model updates to application restarts or to a lazy-reload mechanism. For the quantitative hedge-fund target market — where request volumes are modest, models are updated daily or weekly rather than continuously, and latency requirements are in the tens of milliseconds rather than sub-millisecond — this coupling is acceptable.

A lazy-reload mechanism watches for `ModelPromoted` events on the event bus and triggers a per-process reload of the promoted model, with the old version retained in memory until the in-flight requests drain.

9.4.3 High-throughput or GPU-bound serving

When a specific model's serving requirements exceed what the main application process can handle — GPU inference, QPS beyond a few hundred, or strict isolation — the same container image is deployed as a dedicated Cloud Run service with an environment variable selecting a serving-only entrypoint. The routing URL for that model is updated in the application's model registry, and subsequent requests flow directly to the dedicated service.

No separate codebase, no separate build pipeline, no separate deployment — only a second Cloud Run service running the same image with a different entrypoint.

9.4.4 Inference auditability

Every inference request and response is logged to an `inference_log` table with timestamp, user, model name, model version, input feature hash, output, and latency. This log is the auditable trail that connects a business decision (informed by an inference) back to the model, training data, and code that produced it.

For regulated customers, the inference log is retained alongside the event store with the same durability guarantees and is exportable on demand.

9.5 Feature extraction reuse

Features used at training time must match those used at serving time — the classic train-serve skew problem. The platform addresses this by packaging feature extraction code into the `pyfunc` wrapper of each registered model. The same Python callable that computes features from a silver-layer row at training time is invoked at inference time.

For customers who need a full feature store (shared features across models, online/offline sync, time-travel feature queries), Feast is the portable, fully-local-compatible recommendation and can be layered on without disturbing the rest of the stack. It is not in the default scope because quantitative hedge funds typically maintain their feature pipelines inside their model code rather than externalising them.

Chapter 10

Infrastructure

10.1 Per-tenant resource topology

A single customer instance consists of a dedicated GCP project containing the resources shown below. Every resource is provisioned and managed by a Terraform module, applied as a unit per tenant.

Resource	Purpose
GCP project	Tenant isolation boundary, billing boundary, IAM boundary
Cloud Run service (`api`)	The API role of the application image
Cloud Run services (`worker-*`)	One Cloud Run service per worker role, each stamped from the same image with a different `ROLE` env var
Cloud Run service (`scheduler`)	APScheduler daemon, single replica (min=1 max=1)
Cloud Run service (mlflow)	MLflow tracking server (separate image)
Cloud SQL (default) or AlloyDB (where required) instance	Postgres with AGE, TimescaleDB, PGMQ, pg_cron extensions. Cloud SQL is the default; AlloyDB is selected only when a required extension is unavailable on Cloud SQL in the deployment region. See Architecture chapter §Container view for the rule
GCS bucket (data)	Bronze layer files, exported datasets, scheduled inference outputs
GCS bucket (artefacts)	MLflow model artefacts, training checkpoints
GCS bucket (frontend)	Optional: React SPA assets for CDN delivery
Artifact Registry	Pulls only; images are pushed from the vendor's shared registry
Secret Manager entries	IdP configuration, session JWT signing key, data source credentials
Cloud Load Balancing	Custom domain termination, SSL, Cloud CDN for frontend
VPC Service Controls perimeter	Optional: additional egress controls for regulated tenants

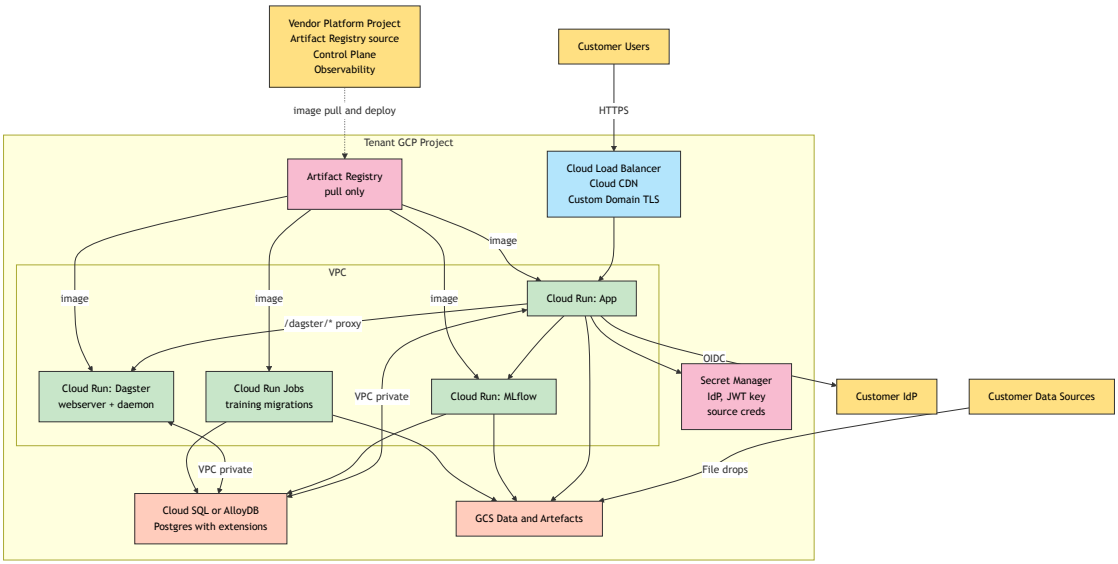

Figure 10.1: Per-tenant infrastructure layout

All application Cloud Run services run the **same image**. They differ only in their `ROLE` environment variable, their autoscaling policy, and their service account bindings. This is the single-image-multi-role pattern described in the application architecture chapter, realised at the infrastructure layer.

Billing is naturally attributed at the project level. Monitoring and logging scope to the project. IAM grants are scoped to the project's service accounts. Quotas are per-project.

10.2 Worker autoscaling on queue depth

The defining operational property of the worker roles is that they must autoscale on PGMQ queue depth. A graph projector pool with a ten-thousand-message backlog must scale out even if individual workers have low CPU; the bottleneck is parallelism, not per-worker capacity. When the queue drains, the pool must scale back in to avoid paying for idle capacity.

Cloud Run's native autoscaling is driven by HTTP request rate, which is not a direct readout of queue depth. Three implementation options close this gap on GCP; the blueprint supports all three but defaults to the first.

10.2.1 Option A: PGMQ-to-HTTP bridge (default)

A thin bridge process reads from PGMQ and issues an HTTP POST per message to the worker's Cloud Run endpoint, which processes the message and acknowledges. Cloud Run's request-rate autoscaling then behaves as it does for a Pub/Sub-push-backed Cloud Run service: requests come in, replicas scale up; requests stop, replicas scale down.

The bridge runs as its own role (`bridge-pgmq-http`) inside the same image, deployed as a small Cloud Run service with `min_instances=1` and modest concurrency. Its responsibility is narrow: drain PGMQ, post to the worker, handle HTTP errors by letting PGMQ's visibility timeout expire and retry.

This keeps everything on Cloud Run with no Kubernetes in the stack. It is the simplest option and the default in the Terraform module. The trade-off is one additional small service per queue; in practice a single bridge replica can drive many queues by multiplexing.

10.2.2 Option B: KEDA on GKE Autopilot

For larger deployments where queue-depth autoscaling fidelity matters more than infrastructure simplicity, worker roles run on GKE Autopilot with KEDA (Kubernetes Event-Driven Autoscaling). KEDA has a native Postgres scaler that queries PGMQ's queue-depth view on a configurable interval and scales the deployment accordingly. Latency from queue depth to replica count is lower than Option A; the ceiling on replicas is higher.

The cost is a Kubernetes cluster to operate. GKE Autopilot removes most of the node-management overhead, but the mental model — pods, deployments, HPAs, service accounts with Workload Identity — is additional. This option is offered as a scaling path, not as the default.

10.2.3 Option C: custom scaler modifying Cloud Run `min_instances`

A periodic job reads queue depth from PGMQ and calls `gcloud run services update` (or the REST API equivalent) to raise `min_instances` when backlog grows and lower it when

backlog drains. Intermediate fidelity — not as responsive as KEDA, not as smooth as Option A, but keeps everything on Cloud Run and needs no additional service per queue.

Offered as an option for teams who find Option A's additional service unattractive but who do not want to introduce GKE. Rarely chosen in practice.

10.3 Training and large-compute workloads

Training jobs — particularly GPU or multi-hour CPU jobs — do not fit the Cloud Run request-response model. They run as Cloud Run Jobs (for CPU, up to 24 hours) or as Vertex AI Custom Training (for GPU). Both are dispatched by the `worker-training` role and tracked asynchronously through PGMQ status-polling messages.

A Cloud Run Job is itself a Cloud Run resource stamped from the same image. The `ROLE` env var for a training job is `job-training`, and the job's entrypoint reads the training-run configuration from its launch arguments rather than from a queue.

10.4 Cross-tenant resources

The vendor operates a small number of shared resources in a dedicated platform project:

- **Artifact Registry** — the single source for application container images. Each tenant's Cloud Run service pulls from this registry. Images are tagged by semantic version and by git commit SHA.
- **Control plane service** — the vendor's internal fleet-management application (see below).
- **Observability aggregator** — Cloud Logging and Cloud Monitoring scoping projects that aggregate signals across all tenant projects for vendor-side oncall.
- **GitHub Actions runners** — standard hosted runners, authenticated to tenant projects via Workload Identity Federation.

10.5 The control plane

The control plane is the vendor's internal tool for managing the fleet of tenant instances. It is itself a small application — a FastAPI process with a Postgres database, deployed to Cloud Run in the platform project — and it is the operational nerve centre of the product.

10.5.1 Responsibilities

- **Tenant registry** — the authoritative list of tenants, their configurations, their deployment topology (silo vs BYOC), their current application version, and their maintenance windows.
- **Provisioning orchestration** — triggering Terraform applies for new tenants, tracking the progress of provisioning steps, surfacing failures to the vendor's operations team.
- **Upgrade orchestration** — coordinating version rollouts across the fleet, respecting per-tenant maintenance windows and canary sequencing.
- **Per-tenant telemetry** — aggregated dashboards showing health, usage, and billing signals for each tenant.
- **Fleet telemetry** — cross-tenant views for capacity planning, error rate trends, and version sprawl tracking.

- **Customer contact and contract metadata** — for integration with support tooling, billing systems, and renewal workflows.

The control plane has no direct customer access. Customers interact with their own instance; the control plane is an internal product for the vendor's operations function.

10.5.2 Silo vs BYOC

The control plane supports both tenancy deployment models:

Silo (vendor-hosted): the tenant GCP project lives in the vendor's GCP organisation. The vendor has full IAM access, operates the infrastructure, and bills the customer through the vendor's pricing plan. The control plane drives Terraform directly.

BYOC (customer-hosted): the tenant GCP project is owned by the customer. The customer grants the vendor a limited-scope service account with permissions to deploy Cloud Run revisions, read Cloud Monitoring metrics, and run Cloud Run Jobs for migrations. All data stays within the customer's cloud perimeter. The control plane drives Terraform through the customer-granted identity; certain operations (project creation, organisation-level policy changes) remain with the customer.

The Terraform module is identical across both models. Only the execution identity changes.

10.6 Terraform module structure

A single root Terraform module provisions a tenant. Its inputs are tenant-specific: name, region, tier, IdP configuration, custom domain, data source credentials. Its outputs are the application URL, the MLflow URL, and the Cloud SQL connection string.

```
terraform/
├── modules/
│   ├── tenant/              # the root per-tenant module
│   │   ├── main.tf
│   │   ├── variables.tf
│   │   ├── outputs.tf
│   │   ├── project.tf       # GCP project creation
│   │   ├── network.tf       # VPC, subnets, firewall
│   │   ├── database.tf    # Cloud SQL (default) or AlloyDB (where required); see Architec
│   │   ├── app.tf           # Cloud Run services
│   │   ├── storage.tf       # GCS buckets
│   │   ├── iam.tf           # service accounts, IAM bindings
│   │   ├── secrets.tf       # Secret Manager entries
│   │   └── dns.tf           # custom domain, load balancer
│   ├── platform/            # shared platform resources
│   │   ├── artifact_registry.tf
│   │   ├── control_plane.tf
│   │   └── observability.tf
│   └── control_plane_db/
├── tenants/
│   └── {tenant-id}/
│       ├── terraform.tfvars # tenant-specific values
│       └── backend.tf      # remote state (GCS bucket in platform project)
└── platform/
```

```
└── terraform.tfvars     # platform-wide configuration
```

Each tenant has its own Terraform state file in a GCS bucket in the platform project. The control plane invokes `terraform apply` with the appropriate working directory and state path when provisioning or upgrading a tenant.

10.7 Secrets management

All secrets live in Secret Manager — one entry per secret, per tenant. Naming convention: `{tenant-id}/{secret-name}`. The application container reads secrets at startup via the application's service account, which has `roles/secretmanager.secretAccessor` scoped to the project.

Secrets include:

- **`idp-client-secret`** — the OAuth client secret for the customer's identity provider
- **`session-jwt-signing-key`** — the signing key for session JWTs (rotated quarterly)
- **`data-source-credentials/{source}`** — credentials for each configured data source
- **`database-password`** — Cloud SQL connection password (Cloud SQL IAM auth is preferred where supported, which eliminates this entry)

No secrets exist in container images, Terraform state files (marked sensitive), or GitHub Actions variables. Rotation is a Terraform operation that writes a new version to Secret Manager; the application picks up new values on its next startup, which is triggered by a Cloud Run revision deploy.

10.8 Network posture

Each tenant project has a dedicated VPC. Cloud Run services use Direct VPC Egress to reach Cloud SQL and any internal resources. Ingress is through Cloud Load Balancing with a managed SSL certificate bound to the customer's custom domain.

For tenants with strict egress requirements, VPC Service Controls are configured to restrict the Cloud Run service to specific APIs and specific external destinations. For BYOC tenants, the customer typically enforces their own VPC Service Controls perimeter, and the vendor's deployment service account operates within that perimeter.

10.9 Regional deployment

Each tenant is deployed to a single region (typically the region closest to the customer or dictated by their data residency requirements). Multi-region deployments within a tenant are not supported by the default topology and would require substantial additional complexity (Cloud SQL replicas, cross-region application deployment, DNS-based routing) that is not justified by the target market's latency expectations.

Disaster recovery is handled by Cloud SQL point-in-time recovery (backed up continuously, restorable to any point within the retention window) and GCS cross-region replication for bronze data (optional, enabled per tenant's disaster recovery requirements).

Chapter 11

Local Development

11.1 The golden rule

Every production behaviour must be reproducible on a single developer laptop. This is not a target; it is a constraint. Any cloud dependency that cannot be simulated locally is disqualified from the architecture. This constraint is the reason PGMQ replaces Pub/Sub, APScheduler replaces Cloud Scheduler, authlib replaces Identity Platform, and Cloud Workflows is entirely absent from the stack.

The practical test: a new engineer with a reasonably-specced laptop, a fresh checkout of the repository, and no GCP credentials should have a running local instance with seeded data, live authentication, functioning pipelines, and a working model training path in under fifteen minutes. Any PR that breaks this test does not merge.

11.2 The docker-compose stack

Four containers cover the full production dependency set:

1. **`postgres`** — Postgres 16 with AGE, TimescaleDB, PGMQ, and pg_cron extensions pre-loaded. Custom image built from a simple Dockerfile that layers the extensions onto the official Postgres image.
2. **`minio`** — S3-compatible object storage, used as the GCS substitute. The application's storage adapter selects MinIO when the `STORAGE_BACKEND` environment variable is set to `minio`.
3. **`mock-oidc`** — a lightweight OIDC provider simulating the customer's Google Workspace or Entra directory. Either a purpose-built container (the `oidc-provider-mock` image is a common choice) or a tiny FastAPI application maintained in the repository for full control.
4. **`mlflow`** — the MLflow tracking server, backed by the same Postgres instance (distinct schema) and using MinIO for artefact storage.

The application itself runs outside docker-compose during development, under `uvicorn --reload`, so that code changes propagate instantly. For CI and for reproducibility-sensitive integration tests, a fifth `app` service is included in an alternate compose file.

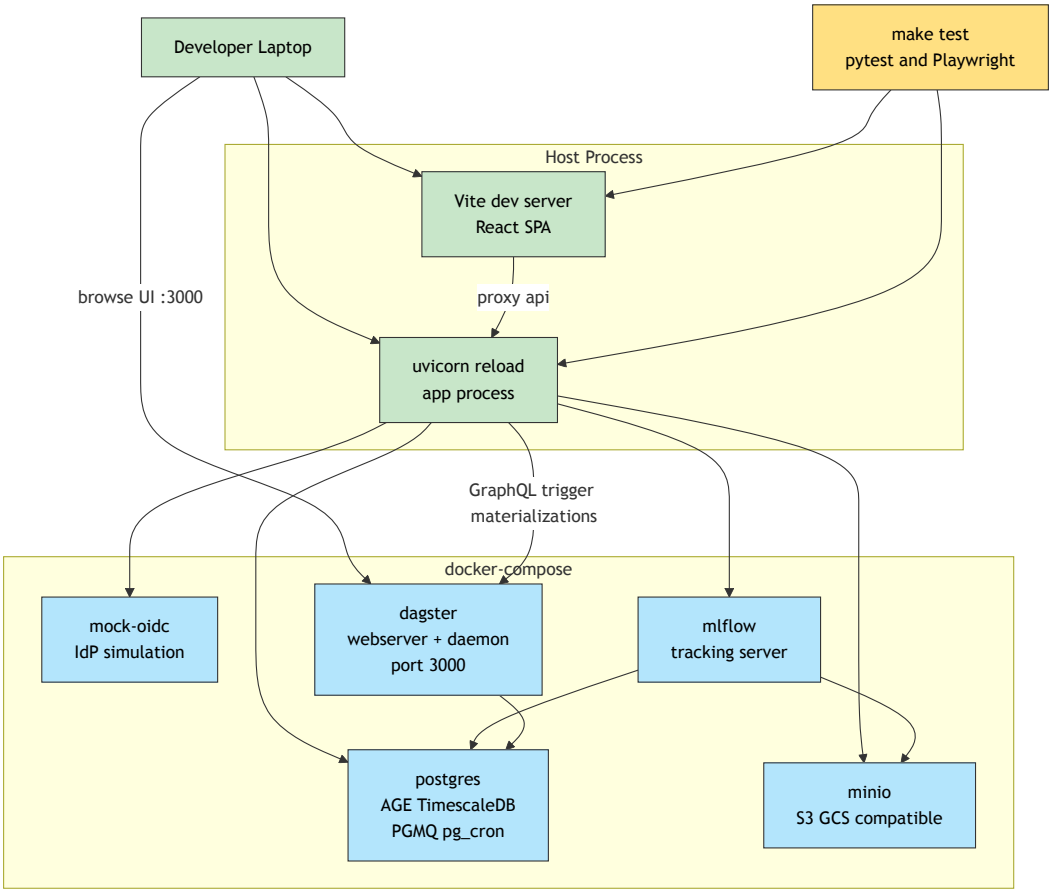

Figure 11.1: Local development topology

11.3 Emulator conventions

The application code uses environment variables to select between production and local backends. The major Python libraries in the stack all respect well-known emulator conventions:

Library	Variable	Effect
`google-cloud-storage`	`STORAGE_EMULATOR_HOST`	Routes all GCS calls to the MinIO endpoint
`google-cloud-pubsub`	`PUBSUB_EMULATOR_HOST`	Not used — PGMQ is used instead
`mlflow`	`MLFLOW_TRACKING_URI`	Points at the local MLflow container
`authlib` (custom)	`OIDC_DISCOVERY_URL`	Points at the mock OIDC provider

A single `.env` file in the repository root holds development values. Production overrides come from Secret Manager, injected via Cloud Run environment variables. The application code never has conditional logic like `if environment == "local"` — it reads configuration and trusts it.

11.4 The Makefile

All common development operations are Makefile targets, giving engineers a single vocabulary regardless of the underlying tooling:

Target	Purpose
`make dev`	Bring up the docker-compose stack in the background
`make dev-stop`	Stop the docker-compose stack
`make migrate`	Apply Alembic migrations against the local Postgres
`make seed`	Populate the database with test tenants, users, and sample data
`make run`	Start the application under `uvicorn --reload`
`make test`	Run the full test suite (unit + integration) against the local stack
`make test-unit`	Run only unit tests (no stack required)
`make test-e2e`	Run Playwright end-to-end tests against the local stack
`make proto`	Regenerate protobuf stubs after schema changes
`make openapi`	Regenerate the frontend API client from the OpenAPI spec
`make lint`	Run ruff, pyright, and frontend linters
`make format`	Auto-format Python and TypeScript
`make clean`	Remove build artefacts and stop running services

New engineers rely on this vocabulary. Senior engineers extend it when new operations become routine. The Makefile is canonical, version-controlled, and tested in CI.

11.5 Seed data

The `make seed` target populates the local database with a reproducible set of tenants, users, models, training runs, and time-series data. Seed data lives in the repository as parquet files and SQL fixtures, not as generated data, so that it is stable across test runs and across developers.

Typical seeds include:

- Two tenants (a healthy one and a degenerate one for edge-case testing)
- A half-dozen users per tenant spanning the role vocabulary
- Three registered models in MLflow with multiple versions each
- Thirty days of synthetic market data in the silver layer
- A handful of failed and successful training runs for status-display exercise
- Inference log entries demonstrating the audit trail

The same seed data is used in integration tests, in Playwright end-to-end tests, and in the demo environment. This reuse ensures that UI layouts, API contracts, and pipeline behaviour are all validated against a consistent fixture set.

11.6 Testing strategy

The test pyramid has three layers:

11.6.1 Unit tests

Target the domain layer. No I/O, no database, no network. Tests run in tens of milliseconds; the full unit suite runs in under thirty seconds. Coverage goal: every non-trivial business rule has at least one positive and one negative test case.

Frameworks: `pytest`, `hypothesis` for property-based tests where appropriate.

11.6.2 Integration tests

Target the application's HTTP surface and its persistence behaviour. Use FastAPI's `TestClient` to issue requests against the real application, backed by the docker-compose Postgres, PGMQ, MinIO, and mock OIDC provider. Each test runs in its own database transaction, which is rolled back at the end.

This is the layer that validates that the application actually works end-to-end within a single machine. Every feature has integration tests; every bug fix is accompanied by an integration test that fails before the fix.

11.6.3 End-to-end tests

Target the user-facing flows through the React UI, driven by Playwright. Run against the same docker-compose stack, with the application running under uvicorn and the frontend running under Vite's dev server. These tests are fewer in number (dozens, not thousands) but exercise real user journeys: login, model submission, training-run monitoring, inference invocation.

Because the entire stack is local, Playwright tests run on every PR in CI without requiring ephemeral cloud environments.

11.7 CI parity

The CI pipeline (GitHub Actions) runs the identical docker-compose stack. Integration tests in CI are not running against a staging environment or ephemeral cloud resources — they are running against the same containers that engineers use locally. This ensures that "works on my machine" and "works in CI" diverge only in obvious, diagnosable ways (resource constraints, GitHub runner flakiness) rather than in subtle behavioural differences.

The CI job that executes `make test` is the single gate on merging to `main`. It runs in approximately five to eight minutes on a standard GitHub-hosted runner.

11.8 Onboarding

The onboarding experience is deliberately polished. A new engineer's first day runs as:

1. Clone the repository.
2. Run `make setup` — uses `uv sync` to install Python dependencies (seconds, not minutes), installs frontend dependencies with `npm ci`, and builds the custom Postgres image.
3. Run `make dev` — brings up the stack.
4. Run `make migrate && make seed && make run` — working application.
5. Open `http://localhost:8000` — log in as a seeded user.

Total elapsed time, assuming dependencies cached: under ten minutes. If any step of this experience takes longer or requires manual intervention, it is treated as a bug and fixed.

This experience is not separate from production readiness — it *is* production readiness. The rigour required to make local development work well is the same rigour that keeps production deployments boring.

11.9 Tooling foundations

The development tooling is chosen for speed and for consistency between local, CI, and container builds:

- **uv** (Astral) manages Python dependencies and virtual environments. `uv.lock` pins exact versions; `uv sync` produces identical environments everywhere. Package resolution and installation are measured in seconds rather than minutes, which materially shifts the feel of the edit-test cycle.
- **ruff** (Astral) handles formatting and linting in a single Rust binary. `make lint` runs in under a second on the full Python codebase. It replaces the conventional stack of black, isort, flake8, pyupgrade, and pydocstyle.
- **pyright** (or Astral's **ty** once generally available) enforces static typing. CI fails on type errors; the local `make lint` target runs the same check.
- The frontend uses **Vite** for the dev server and production build, **eslint** with the flat-config defaults, **prettier** for formatting, and **Vitest** for unit tests.

Every tool listed here is also used in the CI pipeline with the same configuration and the same version pinned in `uv.lock` or `package-lock.json`. "Works on my machine; fails in CI" is

eliminated by making the machine and CI use identical toolchains.

Chapter 12

CI/CD and Deployment

12.1 Pipeline overview

Every change flows through a single continuous-delivery pipeline hosted on GitHub Actions. The pipeline is driven entirely by repository events: pull-request creation, merge to `main`, tag creation, and manually-dispatched workflows for operational tasks.

The pipeline authenticates to Google Cloud using Workload Identity Federation. No long-lived service account JSON keys exist in GitHub secrets, in the repository, or anywhere else. The federation pool and provider are provisioned once, per GCP organisation, as part of the platform Terraform module.

12.2 Runners — hosted and self-hosted

The pipeline uses two classes of runner with deliberately different trust postures.

GitHub-hosted runners (`runs-on: ubuntu-latest`) execute every job that processes untrusted code: PR checks, integration tests, image builds for PR preview, security scans. These jobs receive no secrets that could be exfiltrated from a fork PR. They build artefacts and push them to Artifact Registry using short-lived Workload Identity Federation tokens minted for the specific run.

Self-hosted runners execute every job that deploys an already-built image to a specific target environment. Self-hosted runners sit inside the target network and have the privileges required to operate on it — Docker socket access on a dev host, network access to a private Cloud SQL instance, VPN routes into a customer's on-premises environment. These runners never receive fork PR events; their workflows are restricted to the `main` branch and to `workflow_dispatch` triggers.

The dev/staging deployment target for this blueprint is a Docker Desktop host on an internal Windows server. A self-hosted runner installed on that host executes the dev deployment workflow; the runner has direct access to the local Docker daemon and pulls images from Artifact Registry using a service-account key scoped to `roles/artifactregistry.reader`. Production GCP deployments continue to use GitHub-hosted runners with Workload Identity Federation, because the target — Cloud Run in a customer's project — is reachable over the public internet with IAM-authenticated calls.

12.2.1 Runner labelling

Self-hosted runners carry labels that workflows target explicitly:

Labels	Target
`self-hosted, windows, docker-desktop, dev`	Dev Docker host on the internal Windows server
`self-hosted, linux, gke, staging`	Optional: GKE-resident runner for staging (used only when KEDA path is active)
`self-hosted, linux, byoc, tenant-{id}`	Per-tenant BYOC runner, provisioned in the customer's environment

A workflow declares `runs-on: [self-hosted, windows, docker-desktop, dev]` to pin to the dev runner. Multiple runners can share the same labels for horizontal scaling; GitHub's scheduler picks an idle runner.

12.2.2 Security posture for self-hosted runners

Self-hosted runners are a materially different trust boundary from hosted runners. The blueprint applies four controls:

1. **Branch restriction** — workflows using self-hosted runners trigger only on `push` to `main`, on tag, or on `workflow_dispatch`. Never on `pull_request` from untrusted sources.
2. **Repository-level only, not organisation-level** — the runner is registered to the specific repository, not to the org. This prevents any other repository in the org from accidentally scheduling a job on it.
3. **No secrets in repo** — secrets accessed by self-hosted runner jobs are fetched at runtime from the runner's own environment (Secret Manager via a workload identity on the runner host, or environment variables set by the runner service on startup). They do not flow through GitHub Actions secrets.
4. **Ephemeral execution** — the runner is configured for single-use ephemeral jobs where possible, so a job cannot leave state that affects a subsequent job.

12.2.3 Installation on the dev Windows host

To install the self-hosted runner on the Windows Docker host:

1. On GitHub, navigate to **Repository Settings -> Actions -> Runners -> New self-hosted runner**. Select Windows, x64.
2. On the Windows host, open PowerShell as Administrator in the target directory (for example `C:\actions-runner`).
3. Follow the commands GitHub provides — `Invoke-WebRequest` to download the runner, `./config.cmd` to register it, and `./svc.sh install` / `./svc.sh start` to run it as a Windows service. Apply the labels `self-hosted,windows,docker-desktop,dev` during registration.
4. Grant the runner service account access to the Docker Desktop user's named pipe. The simplest path is to install the runner service as the same Windows account that runs Docker Desktop; alternatively, add the runner's service account to the `docker-users` local group.
5. Verify by triggering a workflow with `runs-on: [self-hosted, windows, docker-desktop, dev]`; the job should start on the Windows host within seconds.

Maintenance of the runner is an operational responsibility. The runner auto-updates its agent binary; the Windows host, Docker Desktop version, and installed tooling (Python, Node) are the operator's responsibility.

12.3 Pipeline stages in detail

The pipeline is a graph of stages, each running in an isolated GitHub Actions job. Stages that are independent run in parallel; stages with dependencies run sequentially. A representative concrete pipeline:

12.3.1 Stage 1: Source checkout and toolchain setup

Every job begins by checking out the repository at the triggering commit and preparing the toolchain.

- **Python**: `setup-uv` action installs `uv` at a pinned version; `uv sync --frozen` reads `uv.lock` and produces an identical virtual environment to the one engineers use locally. Caching uses the hash of `uv.lock` as the cache key, so a lockfile change invalidates the cache but nothing else does.
- **Node**: `setup-node` action at a pinned version; `npm ci` from `package-lock.json`. Cache keyed on the lockfile hash.
- **Docker Buildx**: enabled for multi-platform image builds (`linux/amd64` for production, `linux/arm64` for local parity on Apple Silicon).

This stage completes in seconds when caches hit.

12.3.2 Stage 2: Static checks (parallel fan-out)

Three jobs run in parallel:

- **Lint**: `ruff check` across the Python codebase; `eslint` and `prettier --check` across the frontend; `terraform fmt -check` across infrastructure.
- **Type-check**: `pyright --project pyproject.toml` for Python; `tsc --noEmit` for TypeScript.
- **Security scan**: dependency vulnerability scan via `pip-audit` against the uv-resolved environment (to be replaced by a `uv audit` first-party command once Astral ships it); GitHub Dependency Review on the lockfile; secret-pattern scan via `trufflehog` or equivalent.

Any failure here terminates the pipeline. These checks complete in under a minute.

12.3.3 Stage 3: Unit tests

- **Python unit**: `pytest -m unit` — tests in the domain and application-core modules with no external dependencies. Completes in tens of seconds.
- **Frontend unit**: `vitest run` — React component and utility tests. Completes in tens of seconds.

12.3.4 Stage 4: Integration tests

A single job that:

1. Brings up the docker-compose stack (`make dev`, or a CI-specific compose file).

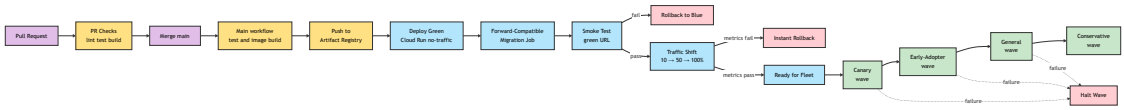

Figure 12.1: Blue/green deployment flow

2. Runs Alembic migrations (`make migrate`).
3. Seeds the database with the canonical test fixtures (`make seed`).
4. Runs the full integration suite (`pytest -m integration`) against the live stack.
5. Tears down the stack on completion.

The integration stage takes the longest of any single job — typically three to six minutes — but it validates the behaviour that matters most: the application working end-to-end against real Postgres, real PGMQ, real MinIO, real MLflow, real mock-OIDC.

12.3.5 Stage 5: Frontend build and E2E tests

- `npm run build` produces the production React bundle.
- The bundle is served alongside the application (either mounted into the FastAPI container or served by Vite preview, depending on the test variant).
- `playwright test` drives the full stack through the UI, exercising the login-to-inference user journey.

Playwright runs headless in CI; engineers can reproduce the same tests locally with `--headed` for debugging.

12.3.6 Stage 6: Container image build

Once all tests pass, a single job builds the application container:

1. Multi-stage Dockerfile: a `builder` stage installs dependencies with `uv sync --no-dev --frozen`, compiles any native code, and builds the frontend bundle. A `runtime` stage copies only the built artefacts onto a minimal base image (Chainguard Python or Distroless).
2. Buildx produces `linux/amd64` (and optionally `linux/arm64`) images.
3. The image is tagged with three labels: the semantic version (`v1.4.2`), the version with commit SHA (`v1.4.2-abc1234`), and the floating tag `main-latest`. Pull-request builds use `pr-{number}` tags instead.
4. The image is pushed to Artifact Registry at `{region}-docker.pkg.dev/{platform-project}/apps/quant-platform`.
5. Artifact Registry's built-in vulnerability scanner runs asynchronously after the push.

12.3.7 Stage 7: Staging deployment (main branch only)

On merge to `main`, the pipeline deploys the new image to the staging tenant. Because the application runs as multiple Cloud Run services stamped from the same image — one per role (`api`, `worker-proj-ui`, `worker-proj-graph`, `worker-pipeline-bronze`, `worker-training`, and so on) — the deployment fans out across every service in the tenant's deployment set.

1. A migration job (`gcloud run jobs execute`) runs Alembic against the staging Cloud SQL instance, applying forward-compatible schema changes.
2. For **each** Cloud Run service in the tenant's deployment set, a new revision is created with `--no-traffic` using the new image tag. The same image URI is used for every service; only the `ROLE` env var differs.
3. Smoke-test jobs run against the new revisions' direct URLs. `api` smoke tests cover authentication, a representative command, a representative query, and a training submission. Worker smoke tests enqueue a test message and assert it is processed.

4. If all smoke tests pass, traffic is shifted in steps across all services (10% -> 50% -> 100%) with a two-minute pause between steps for metric observation. Each step is a `gcloud run services update-traffic` invocation per service; the pipeline parallelises across services.
5. Old revisions are retained for twenty-four hours to support rapid rollback; after that, they are automatically pruned.

The failure of any single service's smoke test or traffic shift pauses the fan-out and reverts traffic on services that have already been shifted. Deployment is transactional at the tenant level, not at the individual-service level — either every service advances or none does.

12.3.8 Stage 8: Production rollout (control-plane-driven)

Production tenant deployments are not triggered directly by the GitHub Actions pipeline. The successful staging deployment emits an event that the control plane receives, recording the image as "available for production." The control plane then orchestrates the wave-based rollout described below, scheduling per-tenant deployments within their maintenance windows.

12.4 Workload Identity Federation concretely

The pipeline never holds a GCP credential. Instead, GitHub's OIDC token is exchanged for a short-lived GCP access token at the start of each job that needs GCP access. The mechanism:

1. Once per GCP organisation, a Workload Identity Pool and a GitHub OIDC Provider are provisioned by Terraform. The provider binds the pool to the GitHub organisation and, optionally, to a specific repository or branch.
2. A service account per environment (`ci-staging@...`, `ci-production@...`) is granted `roles/iam.workloadIdentityUser` on the pool, with an attribute condition matching the expected repository and branch.
3. The service account has the minimum IAM roles for its intended operations: Artifact Registry writer for image-push jobs, Cloud Run deployer for deployment jobs, and so on.
4. In the GitHub Actions workflow, the `google-github-actions/auth` action performs the token exchange and sets up the gcloud CLI and the Google client libraries to use the exchanged credential.

This pattern is the current best practice for GitHub Actions authenticating to Google Cloud and is well documented in the `google-github-actions` repository.

12.5 Workflow file structure

GitHub Actions workflows live in `.github/workflows/` and are organised by trigger:

Workflow	Trigger	Purpose
`pr.yml`	`pull_request`	Full test suite, container build with PR tag, optional preview deploy
`main.yml`	`push` to `main`	Re-runs tests, publishes image with `main-latest`, deploys to staging

Workflow	Trigger	Purpose
`release.yml`	Semantic version tag (e.g. `v1.4.2`)	Publishes release image, notifies control plane
`infra.yml`	Changes under `terraform/`	Terraform plan on PR, apply on merge
`tenant-deploy.yml`	`workflow_dispatch` from control plane	Deploys a specific image tag to a specific tenant
`nightly.yml`	Scheduled	Drift detection, dependency updates, long-running integration tests

Each workflow reuses composite actions in `.github/actions/` — `setup-python`, `setup-frontend`, `gcp-auth`, `build-and-push` — so repetition is minimised and changes to the toolchain touch a single location.

12.6 Pipeline performance expectations

With the toolchain described above, the end-to-end time for a pull-request pipeline is expected to fall in the five-to-eight-minute range on standard GitHub-hosted runners, with the majority of that time spent in the integration and E2E stages. Unit tests and static checks complete in under two minutes combined. Container build is typically one to two minutes.

The main-branch pipeline extends this by approximately five to ten minutes for the staging deploy plus smoke test plus gradual traffic shift. Production tenant rollouts, orchestrated by the control plane, proceed on per-tenant maintenance windows and are measured in hours per wave rather than minutes.

12.7 Failure modes and their handling

- **Test flake** — flaky tests are quarantined (moved to an ignored marker) on first occurrence and tracked in an issue. The quarantine is a forcing function: a flaky test is a broken test and must be fixed or deleted.
- **Transient registry failures** — image push retries are built into the pipeline with exponential backoff.
- **Deploy failure mid-rollout** — automatic traffic reversion to the previous revision; the pipeline is marked failed; the control plane is notified if the failure was on a tenant.
- **Migration failure** — migrations run as a separate job before deployment; a migration failure blocks deployment without leaving the database in an inconsistent state. Forward-compatible migration design means the prior revision continues to work.
- **Smoke test failure** — the green revision is drained; the traffic shift does not proceed; an alert is raised.
- **Post-deployment metric regression** — if the control plane observes elevated error rates after a rollout, it halts the wave and can automatically roll back by re-running the deployment with the prior image tag.

12.8 Pull request workflow

On every pull request, the `pr` workflow runs the following in parallel where possible:

1. **Lint and format** — ruff for Python, eslint and prettier for TypeScript
2. **Type check** — pyright for Python, tsc for TypeScript
3. **Unit tests** — the pytest unit suite
4. **Integration tests** — bring up the docker-compose stack, run the integration pytest suite against it
5. **Frontend build** — `npm run build` producing the static React artefacts
6. **End-to-end tests** — Playwright against the running local stack with the built frontend
7. **Container build** — build the application container image, tag with the PR number and commit SHA, push to Artifact Registry
8. **Security scan** — Artifact Registry vulnerability scan on the pushed image; fail the PR if critical CVEs are detected
9. **Terraform plan** — for any changes under `terraform/`, run `terraform plan` and post the plan as a PR comment

Optional per-PR preview deployments are available: a manually-triggered workflow deploys the PR image to a short-lived Cloud Run revision in the sandbox project and posts the preview URL back to the PR. These are used for UI review and for cross-team demos, not as a substitute for local testing.

12.9 Main branch workflow

On merge to `main`, the `main` workflow:

1. Re-runs the full PR workflow (unit, integration, e2e, container build, scan)
2. Tags the container image as `{semver}-{sha}` and as `main-latest` in Artifact Registry
3. Deploys the new image to the staging tenant instance with zero traffic (a green revision alongside the existing blue)
4. Runs an automated smoke test suite against the green revision's direct URL
5. Shifts traffic to the green revision in increments (10%, 50%, 100%) with health checks between each step
6. If the smoke tests or health checks fail, automatically reverts traffic to the blue revision and marks the deployment failed
7. On success, queues the image for rollout to production tenants per their maintenance windows

Staging deployments are automatic and continuous. Production deployments are coordinated by the control plane and respect per-tenant upgrade preferences.

12.10 Publishing to Artifact Registry

The application's container image is the single deployment artefact. It is built once, on merge to `main`, and promoted unchanged through the environment progression (staging -> canary tenants -> general fleet).

Publishing workflow:

1. The GitHub Actions runner authenticates to GCP via Workload Identity Federation, assuming a service account with `roles/artifactregistry.writer` on the platform project's registry.
2. The image is built with Docker Buildx, targeting both `linux/amd64` and optionally `linux/arm64` for local development parity on Apple Silicon laptops.

3. The image is tagged with the semantic version (`v1.4.2`), the git commit SHA (`v1.4.2-abc1234`), and the floating tag `main-latest`.
4. The image is pushed to `{region}-docker.pkg.dev/{platform-project}/apps/quant-platform:{tag}`.
5. A post-push job triggers Artifact Registry's vulnerability scanner. Critical findings are posted back to the repository as an issue.

Artifact Registry has lifecycle rules that retain the last fifty images tagged with semantic versions indefinitely and delete untagged or PR-tagged images after fourteen days. This keeps registry costs bounded while preserving the ability to roll back to any recent release.

12.11 Blue/green deployment on Cloud Run

Cloud Run's native revision model is the blue/green primitive. Each deploy creates a new revision; traffic is distributed across revisions via an explicit traffic configuration. Rollback is a configuration change, not a rebuild.

A tenant deployment is a **set** of Cloud Run services, one per role, all stamped from the same image. Blue/green applies per service, and the control plane orchestrates the fan-out so that either all services advance to the new image or all revert. The deployment dance, orchestrated by the control plane for each tenant, proceeds as follows — where each numbered step is executed across every service in the deployment set:

1. **Deploy a new revision with zero traffic:**

   ```
   gcloud run deploy {service} \
     --image {registry}/{app}:{version} \
     --region {region} \
     --no-traffic \
     --revision-suffix {version}
   ```

2. **Run migrations** as a separate Cloud Run Job against the tenant's database. Migrations are always forward-compatible — the new revision must work with both the old schema (pre-migration) and the new schema (post-migration). This is enforced by a migration-check test in the PR workflow.

3. **Smoke test the green revision** directly at its unique URL (`{revision}---{service}-{hash}-{region}.a.run.app`). Smoke tests exercise authentication, a representative command path, a representative query path, and the training submission endpoint.

4. **Gradual traffic shift:**

   ```
   # 10% to green
   gcloud run services update-traffic {service} \
     --to-revisions={green}=10,{blue}=90
   # wait, check metrics
   # 50%
   gcloud run services update-traffic {service} \
     --to-revisions={green}=50,{blue}=50
   # wait, check metrics
   # 100%
   gcloud run services update-traffic {service} \
     --to-revisions={green}=100
   ```

5. **Retain the blue revision** for a configured window (typically twenty-four hours) to support rapid rollback.
6. **Emit deployment events** to the control plane's event log: tenant, version, timestamp, outcome.

Rollback at any step is:

```
gcloud run services update-traffic {service} --to-revisions={blue}=100
```

A single command, no rebuild, and traffic shifts in seconds.

12.12 Forward-compatible migrations

Blue/green deployment is only viable when the database schema is forward-compatible — new revisions must not require schema changes that break old revisions. The expand-migrate-contract pattern enforces this:

1. **Expand** — add new columns, tables, or indexes in an additive migration. Both old and new code work.
2. **Deploy new code** that begins using the new schema while continuing to write old fields for the duration of the overlap window.
3. **Backfill** — a separate job populates the new columns from existing data.
4. **Contract** — a later release removes the old columns. This requires all running revisions to be on the new code.

This pattern adds a release cycle to every schema change but eliminates a class of downtime incidents. It is enforced in CI by a check that runs the migration, starts the old revision against the new schema, and exercises a representative test suite. If the old revision breaks, the migration is rejected.

12.13 Fleet upgrade orchestration

Production tenants are on different versions at any given moment — this is a natural consequence of silo tenancy. The control plane tracks per-tenant versions and orchestrates rollouts in waves:

1. **Canary wave** — one or two internal test tenants. Deploy, observe for twenty-four hours.
2. **Early-adopter wave** — customers who have opted into faster releases. Deploy, observe for forty-eight hours.
3. **General wave** — the majority of tenants. Deploy during their maintenance windows over a one-to-two-week period.
4. **Conservative wave** — customers on the slowest track (typically enterprise contracts with strict change control). Deploy after a month of fleet-wide stability on the new version.

Each wave has a defined success criterion (error rate, latency, customer-reported issues). Failure in one wave pauses the rollout and triggers investigation. The control plane enforces these pauses automatically; they are not dependent on human diligence.

12.14 Blue/green across the full stack

The Cloud Run revision-based blue/green covers the application. The full deployment surface also includes MLflow, Cloud SQL (schema migrations), and GCS buckets (immutable data).

- **MLflow** uses the same Cloud Run revision model, with its own blue/green flow. Because MLflow is stateless (state lives in Postgres and GCS), revision rollback is instant.
- **Cloud SQL** migrations are forward-compatible, as described above. Rollback of a migration is avoided; instead, the next release rolls forward to fix any issue.
- **GCS buckets** are immutable — deployment doesn't change bucket contents, only application reads and writes.

12.15 CI/CD for Terraform

Infrastructure changes flow through a parallel pipeline. Terraform code lives in the same repository as the application. On PR, `terraform plan` runs for every changed workspace and posts the plan as a comment. On merge to `main`, `terraform apply` runs automatically for the platform workspace and for non-production tenant workspaces; production tenant applies require a manual approval step in GitHub Actions.

Terraform state is remote (GCS bucket in the platform project, per-workspace). State locking prevents concurrent applies. Drift detection runs nightly and alerts on any unmanaged changes.

12.15.1 Is Terraform overkill?

The question deserves a direct answer rather than reflexive defence of the choice. Terraform adds a configuration language to learn, a state-management concept to maintain, a plan-and-apply discipline to follow, and a class of state-corruption failures to recover from. For a single tenant, a shell script calling `gcloud` commands is legitimately simpler and does the job.

The trade-off tips in favour of Terraform at the point where infrastructure becomes repeatable rather than bespoke. Concretely:

- **One tenant forever, no BYOC, no planned fleet growth.** A `gcloud`-based shell script or a set of Makefile targets is sufficient and arguably preferable. Don't add Terraform for its own sake.
- **Two or more tenants, or any realistic expectation of more.** Terraform's value appears immediately. The module-per-tenant pattern, variable-driven customisation, and remote state make every new tenant a variable change rather than a script rewrite. Drift detection catches manual console edits before they become silent incidents.
- **BYOC at any scale.** Terraform is essentially required. BYOC deployments demand strict, reviewable, auditable definitions of exactly what is being provisioned in the customer's project; customers will ask to see the module, review it, and sometimes run their own plan against it. A shell script is not a defensible artefact in a customer security review.
- **Regulated tenants, SOC 2 ambitions, or any compliance framework.** Terraform's reviewability and state auditability are directly required by every modern compliance framework that examines infrastructure controls. Auditors accept Terraform as evidence; they do not accept "we ran some gcloud commands."

The specific CI-integration patterns with GitHub Actions are three:

- **Direct `terraform` CLI in the runner** — the simplest pattern. A workflow checks out the repo, authenticates to GCP via Workload Identity Federation, runs `terraform init` / `plan` / `apply`, and posts results as PR comments. State lives in a GCS bucket; locking is handled by Terraform's native GCS backend.
- **Terraform Cloud / Terraform Enterprise** — HashiCorp's managed runner service, invoked from GitHub Actions via API calls. Adds cost and an external dependency; pro-

vides a polished UI, run history, and policy-as-code. Worth considering only for teams whose operations involve stakeholders outside engineering who benefit from the UI.
- **Atlantis** — a self-hosted runner for Terraform that integrates with GitHub PR comments. Useful for larger teams with many concurrent infrastructure PRs; overkill for a single-team product.

For the blueprint's default, the direct-CLI pattern is the right choice. It keeps everything in GitHub Actions, avoids a second SaaS dependency, and is trivially substitutable for Terraform Cloud later if the team demands a UI.

12.15.2 Alternatives to Terraform, briefly

Pulumi — defines infrastructure in Python (or TypeScript, Go). Attractive to a Python-first team because it shares the language; potentially unified with application code. Downsides: smaller ecosystem, smaller pool of engineers familiar with it, less-mature support for some GCP services. A reasonable choice for a Python-only team; not the default recommendation because the declarative separation of infrastructure from application code is a feature, not a bug.

Config Connector / KCC — GCP's Kubernetes-style declarative resource API. Excellent if the platform already runs on GKE and wants to manage everything through Kubernetes manifests. Misaligned with a Cloud Run-centric architecture; adds a Kubernetes dependency that does not otherwise exist.

gcloud scripts — a defensible choice only for a single-tenant scenario with no compliance obligations and no planned growth. Revisit immediately when the context changes.

12.16 Release testing: platform tests versus customer-specific tests

A new release must be safe for every tenant it will be deployed to. "Safe" has two distinct meanings, and the testing strategy must address both.

Platform safety — the new release does not introduce bugs in the platform's own behaviour. Every command path still works, every query path still works, schemas migrate correctly, auth still functions, the UI still renders. This property is tested once per release in CI and must pass before any tenant sees the new image.

Tenant safety — the new release does not break anything specific to a particular tenant's configuration, data sources, models, or workflows. A release that is fine on tenant A may break on tenant B because tenant B has a data source that uses a feature the release changed, or a model that depends on a library version the release bumped, or an authorisation configuration the release refactors. This property must be tested per tenant, against tenant-specific state, and is irreducible — a platform-level test suite cannot cover it because the test suite does not know what each tenant cares about.

12.16.1 Platform test suite

The platform test suite runs in CI on every pull request and every merge to `main`. It is entirely tenant-agnostic; it uses seed data that represents a generic customer but does not represent any real customer.

Layer	Coverage
Unit	Domain logic, aggregates, pure functions
Contract	OpenAPI schema, event schema, migration compatibility
Integration	Full REST API against docker-compose stack, real Postgres, real PGMQ, real MinIO, real MLflow
End-to-end	Playwright-driven UI journeys
Performance	Representative load tests against synthetic data
Security	Static analysis, dependency scanning, container image scanning

The platform suite is the gate between "code merged" and "image promoted to Artifact Registry." An image that has not passed the full platform suite cannot be deployed to any tenant.

12.16.2 Tenant smoke tests

Each tenant has a smoke-test suite exercising the tenant's specific configuration at the API level. Smoke tests are small — seconds to a few minutes — and cover:

- Authentication against the tenant's configured identity provider (using a dedicated service-account-style test identity)
- A representative command invocation (e.g. submit a training run for a production model)
- A representative query invocation (e.g. fetch the model list, fetch the last inference result)
- File-based ingestion path (drop a small test file, confirm it reaches gold)
- File-based export path (trigger an export, confirm it appears in the expected location)

Smoke tests run automatically after every tenant deployment as part of the blue-green rollout — the green revision is smoke-tested against its direct URL before any production traffic is shifted. A failing smoke test reverts the traffic shift and raises an alert.

Smoke tests are maintained in the platform repository, parameterised by tenant configuration. Adding a smoke test is a platform engineering activity; adjusting it for a specific tenant's quirks is a configuration change.

12.16.3 Customer-specific regression tests

Above smoke tests, customers (particularly enterprise-tier customers) own a test suite that exercises their specific workflows, data sources, and models in ways the platform suite cannot anticipate. These tests are:

- **Provided by the customer** in a repository the platform can execute against their staging environment
- **Executed by the platform** on every upgrade to their staging instance, before promotion to their production silo
- **Blocking** — a failure pauses the customer's upgrade; the customer must approve proceeding or ask for a fix

This layer is the explicit contract between the platform and the customer: the platform promises not to break what the customer has tested. The customer promises to maintain the test suite as their usage evolves. Neither party tests for the other.

Customers who do not maintain a test suite are on their own smoke-test coverage and implicitly accept the risk of tenant-specific regressions. Most enterprise customers build such suites within their first quarter on the platform; the platform supplies templates and runs the tests in their staging environment.

12.16.4 Upgrade-wave testing

The wave-based rollout (canary -> early adopter -> general -> conservative) is itself a testing layer. Each wave produces observable signals:

- Error rates and latency on the rolled-out tenants
- Customer-reported issues
- Smoke-test failures
- Customer-owned regression suite failures

A wave that produces any negative signal halts the rollout. The next wave does not begin until the signal is resolved. This structure means that a regression in a rare configuration — one that is missed by both the platform suite and the customer's own suite — is caught on a small population before reaching the majority of the fleet.

12.16.5 What not to test

Platform CI should not execute customer-specific tests. Customer staging environments should not execute platform CI suites. The two layers are deliberately separate: the platform tests that its invariants hold; the customer tests that its workflows work. Mixing them produces a test suite that is slow, fragile, and expensive to maintain, without improving the safety property either layer provides on its own.

The platform's responsibility ends at the contract it publishes (REST API, file contracts, event schemas, UI surface). The customer's responsibility begins at how they use that contract. Test each on its own side of the line.

Chapter 13

Observability and Operations

13.1 Principles

Observability is built on three pillars — structured logs, metrics, and traces — emitted by the same application code in every environment. The local docker-compose stack produces logs in the same JSON format as production; metrics are emitted against the same schema; traces follow the same propagation conventions. A developer can read production output and a local run side-by-side and compare them directly.

Every signal carries the tenant identifier as a label. In a silo topology, the tenant identifier is the project identifier, automatically attached by Cloud Logging and Cloud Monitoring. No application-level tenant injection is required for production signals; locally, a dev tenant label is set explicitly.

13.2 Structured logging

The application emits every log event as a single-line JSON object with consistent fields:

- `timestamp` — ISO-8601 with microsecond precision
- `level` — `debug`, `info`, `warn`, `error`, `critical`
- `event` — the event name (e.g. `command.dispatched`, `projection.applied`, `training.submitted`, `inference.served`)
- `user_id`, `user_roles` — from the authenticated request context
- `request_id` — propagated from the inbound HTTP header or generated on ingress
- `trace_id`, `span_id` — from the OpenTelemetry context
- `duration_ms` — where meaningful
- `tenant_id` — for local and non-silo contexts; implicit at the project level in production

Free-text log lines are disallowed. Every log line carries structured data. Error logs always include the exception type, message, and stack trace in dedicated fields, not concatenated into a message string.

Cloud Logging parses the JSON format natively and indexes the top-level fields. Queries filter by `event`, `user_id`, `duration_ms`, or any combination. Saved queries (log-based metrics) are the primary alerting mechanism for application-level signals.

13.3 Metrics

The application exposes an OpenTelemetry metrics endpoint scraped by Cloud Monitoring (via the OpenTelemetry Collector sidecar). Metrics fall into three tiers:

13.3.1 Golden signals per endpoint

Every REST endpoint has automatic coverage for:

- **Request rate** (RED: Rate)
- **Error rate** (RED: Errors, by HTTP status and by exception type)
- **Latency distribution** (RED: Duration, histogram with p50, p95, p99)

These are derived from a single FastAPI middleware that observes every inbound request; no endpoint-specific instrumentation is required.

13.3.2 Business metrics

Named counters and gauges tracking domain-level signals:

- Commands dispatched per type
- Events appended per aggregate type
- PGMQ queue depths
- Pipeline runs started, completed, failed (per layer, per source)
- Model training runs submitted, completed (per model, per status)
- Model inference requests (per model, per version)
- Model inference latency distribution
- Data quality violations per source

13.3.3 Infrastructure metrics

Provided by Cloud Run, Cloud SQL, and GCS out of the box. The application does not re-emit these; it relies on GCP's native collection.

13.4 Distributed tracing

The application is instrumented with OpenTelemetry. Every inbound HTTP request opens a root span; every outbound call (database query, PGMQ operation, GCS access, MLflow call) produces a child span. Trace context is propagated via W3C Trace Context headers.

Traces are exported to Cloud Trace. A request that touches the API, the event store, a PGMQ enqueue, a projector handler, a graph update, and an inference call produces a complete trace spanning all components — across the small set of single-purpose worker services (`api`, `worker-proj-*`, `worker-pipeline-*`, `worker-training`, `worker-inference-batch`, `scheduler`) all sharing the same database, which means traces are usually short, the inter-service hops are few, and the topology is easy to reason about.

Cross-service traces (application -> MLflow -> back) include the MLflow service's spans, enabling end-to-end latency analysis of model serving paths.

13.5 Dashboards

Two dashboard tiers exist:

13.5.1 Per-tenant dashboards

A Cloud Monitoring dashboard scoped to each tenant's project. Contents:

- **Service health** — Cloud Run request rate, error rate, latency; Cloud SQL CPU, memory, connections
- **Queue health** — PGMQ queue depths, consumer lag, DLQ accumulation
- **Pipeline health** — daily pipeline run success rate, row counts, data quality score trends
- **Model health** — training success rate, inference latency per model, inference error rate
- **Business activity** — active users, commands per minute, unique models served

Enterprise-tier customers have IAM read access to their own dashboards; they can see the same view the vendor oncall sees.

13.5.2 Fleet dashboards

In the platform project, Cloud Monitoring scoping projects aggregate signals across all tenants. Contents:

- Tenant count by version
- Fleet-wide error rate trend
- Worst-performing tenants by latency or error rate
- Upcoming maintenance windows
- Capacity planning signals (storage growth, database size trends)

Fleet dashboards are accessible only to the vendor operations team.

13.6 Alerting

Alerts are configured in Cloud Monitoring with two severity levels:

Page (wakes someone up): - Application error rate above threshold for five minutes - Database unavailable for two minutes - Queue depth above critical threshold for thirty minutes - Training job failures above threshold for one hour - Authentication failures above threshold for five minutes (potential brute force)

Ticket (creates a work item): - Data quality score below threshold for a day - Storage growth projecting against quota within thirty days - Backup verification failure - Version drift across fleet beyond policy

Every alert has a linked runbook in the repository's `docs/runbooks/` directory. Runbooks follow a strict template: symptom, immediate mitigation, investigation steps, resolution, and post-incident actions.

13.7 Audit trail

In addition to operational logs, the platform maintains an application-level audit trail for security- and compliance-relevant events:

- User login and logout
- Role assignment changes
- Model promotion and demotion
- Training data extraction (including the extracted `as_of` timestamp)
- Inference requests (in the `inference_log` table)

- Administrative configuration changes

The audit trail is append-only, stored in a dedicated Postgres table with cryptographic chaining (each row includes the hash of the previous row's content, detecting tampering). For regulated customers, the audit trail is exported nightly to a WORM GCS bucket with Object Lifecycle Lock.

13.8 Incident response

Each tenant incident follows a standard response:

1. **Detection** — an alert fires or a customer reports an issue
2. **Triage** — the oncall engineer identifies the affected tenant (from alert labels) and classifies severity
3. **Mitigation** — follow the runbook for the alert; the first action is usually traffic rollback to the last-known-good revision
4. **Investigation** — structured logs, traces, and the control plane's deployment history provide the full context
5. **Resolution** — deploy a fix through the normal pipeline; emergency fixes follow an expedited review but never skip the PR and staging gates
6. **Postmortem** — every customer-impacting incident produces a postmortem document, reviewed internally and (for significant incidents) shared with the affected customer

The postmortem template is structured and blameless. Its outputs feed the engineering backlog and the runbook library.

13.9 Operational cadence

The operations team runs a weekly fleet review that walks through:

- Incidents in the preceding week (severity, tenant, root cause, resolution)
- Version drift status across the fleet
- Upcoming maintenance windows and planned changes
- Top five tenants by support burden
- Top five tenants by resource consumption
- Backlog of pending tenant onboardings and offboardings

The review drives tactical prioritisation and surfaces systemic issues that merit engineering investment (e.g. a recurring class of incident that a platform change could eliminate).

Chapter 14

Security and Compliance

14.1 Threat model

The primary threat classes the platform defends against:

1. **Cross-tenant data access** — a user or compromised credential for tenant A gaining access to tenant B's data. Mitigated by the silo topology: tenants share no database, no storage, no compute, and no IAM principal. Cross-tenant access requires crossing a GCP project boundary, which is the strongest isolation primitive GCP offers.
2. **Model IP exfiltration** — an attacker extracting trained models or training data. Mitigated by tenant-scoped Secret Manager and GCS bucket IAM, per-tenant service account binding to Cloud Run and MLflow, and BYOC for customers whose threat model excludes vendor-side access.
3. **Identity impersonation** — an attacker assuming another user's identity within a tenant. Mitigated by delegating authentication to the customer's directory (with its MFA, conditional access, and anomaly detection), short-lived session JWTs, and revocation via the `jti` registry.
4. **Supply-chain compromise** — malicious code introduced through a dependency or build artefact. Mitigated by pinned dependency versions (`uv.lock`, `package-lock.json`), Artifact Registry vulnerability scanning on every push, and Software Bill of Materials (SBOM) generation in CI.
5. **Misconfigured IAM** — overly permissive IAM bindings exposing resources. Mitigated by Terraform-driven IAM (no console edits), least-privilege role assignments validated by Policy Analyzer, and a nightly drift check.
6. **Data exfiltration through the application** — a legitimate user exceeding their authorised scope. Mitigated by role-based authorisation at every handler, domain-level permission checks, and the inference log for post-hoc auditability.

14.2 Tenant isolation

The silo tenancy model is the primary isolation boundary. Each tenant has:

- A dedicated GCP project with its own IAM policy, billing account, and quota.
- A dedicated VPC, restricting network paths to the application's own services.
- A dedicated Cloud SQL instance, with its own connection endpoint and credentials.

- A dedicated GCS bucket, with bucket-level IAM excluding cross-tenant access.
- A dedicated Secret Manager namespace, with secrets scoped to the tenant's service account.
- A dedicated Artifact Registry pull permission (images are shared; pull access is per-project).

There is no code path in the application that could traverse tenants; the application runs in a single tenant's context and does not have IAM permissions to reach any other tenant.

14.3 Encryption

All data is encrypted at rest by default using Google-managed encryption keys. Customers with heightened requirements opt into Customer-Managed Encryption Keys (CMEK) backed by Cloud KMS. Key rotation is enforced on a policy schedule; the application reads the current key version from the resource API on each operation.

All data in transit is encrypted via TLS 1.3 at every hop: browser-to-load-balancer, load-balancer-to-Cloud Run, Cloud Run-to-Cloud SQL (IAM auth tokens), Cloud Run-to-GCS (HTTPS), Cloud Run-to-MLflow (HTTPS). Internal VPC-only traffic is encrypted by Google's transparent network encryption.

For BYOC customers, encryption keys are held by the customer in their own KMS. The application is given wrap/unwrap permission but not read permission on the key material itself.

14.4 Secret handling

Secrets never appear in: - Container images (scanned on push to catch accidental inclusion) - Git history (pre-commit hook scans for credential patterns) - Terraform state files (Secret Manager values are marked `sensitive` and referenced, not embedded) - Application logs (a log-sanitisation middleware redacts values matching secret patterns) - GitHub Actions environment variables (WIF replaces all static credentials)

The application's secret-loading layer reads from Secret Manager at startup, caches values in memory for the lifetime of the container, and never writes them to disk or logs. Rotation is achieved by deploying a new revision; the old revision drains with the old secret, and the new revision starts with the new secret.

14.5 Audit requirements

Regulated customers in the hedge-fund segment typically have audit requirements driven by SEC, CFTC, FINRA, or equivalent non-US regulators. The platform supports these through:

- **Immutable event store** — the append-only `events` table is the source of truth for every domain change; it cannot be modified, only appended to. Historical events can be replayed to reconstruct state at any point in time.
- **Bi-temporal data** — corrections to historical data are recorded alongside the original values, with explicit validity ranges. Training data extractions reproduce historical states faithfully.
- **Inference log** — every model inference is recorded with its inputs, outputs, model version, and requesting user. Correlating a trading or risk decision to its informing inference is a single query.

- **Cryptographically-chained audit trail** — the application-level audit log uses per-row hash chaining to detect tampering. A tamper check is part of the daily backup verification.
- **WORM retention** — audit logs and inference logs are exported to GCS buckets with Object Lifecycle Lock, making them irrevocable for the retention period (typically seven years).
- **Export on demand** — regulators and customer auditors can request an export; the platform provides a self-service export endpoint that produces a signed, timestamped archive of audit data for a date range.

14.6 Data residency

Each tenant is deployed to a specific region, dictated by the customer's residency requirements. The Terraform module accepts the region as an input and configures Cloud Run, Cloud SQL, and GCS accordingly. Cross-region data movement is disabled by default; enabling it (e.g. for disaster-recovery replication) requires an explicit configuration change and is documented in the tenant's configuration file.

Common residency targets: - **US**: `us-central1` or `us-east4` - **EU**: `europe-west1` (Belgium) or `europe-west3` (Frankfurt) - **UK**: `europe-west2` (London) - **Asia**: `asia-northeast1` (Tokyo) or `asia-southeast1` (Singapore)

For customers with split residency requirements (e.g. a European customer whose London office handles UK-domiciled data and whose Frankfurt office handles EU-domiciled data), the solution is multiple tenant instances, one per jurisdiction, not a single multi-region instance.

14.7 BYOC-specific considerations

BYOC deployments introduce an additional layer of access control: the vendor's deployment service account has access only to the specific resources required for lifecycle operations. Typical BYOC IAM grants are:

- `roles/run.developer` on the tenant's Cloud Run services — for deploying new revisions
- `roles/cloudsql.client` on the Cloud SQL instance — for running migrations
- `roles/artifactregistry.reader` on the vendor's Artifact Registry — for pulling images
- `roles/monitoring.viewer` on the tenant's project — for observability access
- `roles/logging.viewer` on the tenant's project — for troubleshooting

The vendor's service account does not have `roles/editor` or `roles/owner` on the customer project. It cannot create new resources, delete existing resources, or modify IAM policies. Structural changes (adding a new service, changing a network boundary) require customer-side action.

The customer provides VPC Service Controls boundaries, organisation policies, and audit log retention on their own terms. The vendor's deployment service account operates within those constraints.

14.8 Vulnerability management

Every dependency is tracked and scanned:

- **Python dependencies** — `uv.lock` pins exact versions; GitHub's Dependabot proposes upgrades for security advisories.
- **JavaScript dependencies** — `package-lock.json` pinning and the same Dependabot coverage.
- **Container base image** — a Distroless or Chainguard Python image, minimising attack surface and patched upstream.
- **Container image scanning** — Artifact Registry's built-in scanner flags vulnerabilities on every push; critical findings block deployment.
- **Terraform modules** — pinned to specific registry versions; upgrades reviewed as regular PRs.

A security review of the SBOM runs on a monthly cadence. The output feeds the engineering backlog.

14.9 Incident disclosure

Security incidents follow an accelerated version of the normal incident response process. The runbook includes a mandatory customer notification step for any incident classified as data-impacting or authentication-bypassing, with a time-to-notification commitment (typically twenty-four to seventy-two hours) documented in the master services agreement.

Customers in regulated industries often require contractual disclosure obligations; these are negotiated per contract and implemented in the platform's incident playbook rather than in code.

Chapter 15

Comparison to Alternatives

The blueprint exists in a market with credible incumbents and credible alternatives. This chapter is the honest competitive map. It exists in the blueprint (not just in marketing) because every architectural decision in the preceding chapters was made with these alternatives in mind — and a reader evaluating the architecture deserves to know what the architecture was choosing *against*.

The chapter is structured by competitor, with explicit "where they win, where we win, where we punt" framing. The goal is calibration, not advocacy: a reader should finish this chapter understanding not just why this architecture was chosen but also when one of the alternatives would be the right answer.

For the customer-facing positioning document — the one-sentence pitch, the buyer profile, the pricing assumptions — see `blueprint/positioning/2026-04-21-positioning.md`. This chapter is the architecture-level comparison.

15.1 SigTech (the closest direct competitor)

SigTech, founded in 2019 as a spinout from Brevan Howard, is the most direct competitor for the platform this blueprint describes. They are a London-based fintech offering a Python-based cloud quant platform with a backtest engine, curated data, and (since 2025) the MAGIC AI-agent layer. Their public customer base spans hedge funds, asset managers, sovereign wealth funds, and pension funds with combined AUM of over $5T.

Where SigTech wins:

- **Maturity.** Six years of production hardening across $5T of customer AUM is a non-trivial moat.
- **Curated data.** SigTech invests substantially in data curation across equities, rates, FX, commodities, and volatility. A customer adopting their platform inherits that data investment.
- **Brand recognition** in London, EU, and increasingly Asia. Their Brevan Howard heritage carries weight.
- **MAGIC product** for AI agents represents serious R&D investment in the LLM-driven workflow space.
- **Sales motion** matured to enterprise tier with ISO 27001 certification.

Where this architecture wins:

- **Silo + BYOC tenancy.** SigTech is multi-tenant cloud SaaS (per public posture; verify in customer conversations). For a hedge fund that has been told by their security team that data cannot leave their cloud perimeter, SigTech is a non-starter. This architecture's per-tenant GCP project, with optional BYOC into the customer's own GCP organisation, is structurally better-fit for that constraint.
- **Open-source-first stack.** SigTech is proprietary; a customer locked in cannot easily exit. This architecture's stack (Polars, Postgres, MLflow, FastAPI, Vite, Tailwind) is open-source and portable. A customer can in principle pull the deployment in-house.
- **Local-first development.** A SigTech researcher works against SigTech's hosted environment. A researcher on this platform works against a local docker-compose simulation of production. The local loop is faster and the on-laptop development cycle is meaningfully different.
- **Mid-market price point.** SigTech's enterprise pricing puts them out of reach for a meaningful slice of the $500M-$5B AUM funds that this architecture targets.
- **Methodological transparency.** This architecture surfaces PBO, DSR, walk-forward configuration, CPCV, and bi-temporal columns as first-class platform concepts the customer can see and configure. SigTech's equivalents are inside their proprietary engine; less transparent to the customer's compliance review.

Where to punt to SigTech:

- $5B+ AUM funds with deep multi-asset, multi-strategy stacks and the budget for enterprise pricing.
- Customers prioritising managed-everything and willing to send data to a third party's cloud.
- European customers where SigTech's London base is an advantage.

15.2 Domino Data Lab

Domino is the largest generic enterprise MLOps vendor by deployed customer count, with over 20% of the Fortune 100 as customers and $228M in venture funding. They are positioned as a horizontal MLOps platform: their customer base spans Lockheed Martin (defence), Allstate (insurance), Bayer (life sciences). They explicitly compete with Databricks, Dataiku, and DataRobot.

Where Domino wins:

- **Generic MLOps maturity.** The platform has been deployed at scale across many industries; the operational characteristics are well-understood.
- **Brand recognition** in enterprise data-science procurement.
- **Customer-success and forward-deployed-engineer model** for large accounts.
- **Integration with non-Python tooling** (R, SAS, Java) for organisations whose data-science work spans languages.

Where this architecture wins:

- **Quant-native opinionated defaults.** Domino does not surface PBO/DSR; does not enforce walk-forward; does not implement bi-temporal data as a first-class concept. A Domino deployment for quant work would require months of custom integration to reach feature parity with what this architecture provides on day one.
- **Open-source vs. proprietary, including the orchestrator.** Domino is proprietary, and so is its workflow orchestration layer. This architecture's orchestrator is Dagster (open source), with run history in the customer's own Postgres and a UI the customer's quants

and operators see directly. A customer who later wants to leave can take their asset definitions and run history with them.
- **Silo tenancy by default.** Domino's deployment model is per-customer-workspace within their managed offering.
- **Lower price point** appropriate for the mid-market segment Domino does not specifically target.

Where to punt to Domino:

- Generic enterprise data-science teams whose ML workloads span medical imaging, NLP, recommendation systems, supply-chain optimisation, and other non-quant domains.
- Customers already on Domino who want to add quant workflows incrementally rather than switch platforms.

15.3 Palantir Foundry / AIP

Palantir Foundry is a multi-domain enterprise data and analytics platform with a sustained $2.8B in trailing-twelve-month US commercial bookings (Feb 2026). Their AI Platform (AIP), Model Studio (no-code training, GA Feb 2026), and Pipeline Builder collectively cover the data-engineering, model-training, and inference workflow that this blueprint also covers.

Where Palantir wins:

- **Enterprise reach.** Foundry has government, defence, healthcare, manufacturing, and financial-services deployments at massive scale.
- **Multi-domain platform breadth.** A customer adopting Foundry can run quant workflows alongside operations, supply chain, and analytics on the same substrate.
- **AI agent layer (AIP)** with substantial capability and deployment evidence.
- **Forward-deployed-engineer model** for high-value implementations.
- **Recent investment in Model Studio** signals continued commitment to the no-code-ML segment.

Where this architecture wins:

- **Sales motion.** Palantir's customer engagements typically involve forward-deployed engineers, multi-quarter implementations, and seven-figure annual contracts. This architecture is mid-market self-serve trial pilot annual contract; the mid-market customer cannot afford Palantir's engagement model.
- **Quant-native focus.** Foundry is multi-domain by design; this platform is quant by design. The opinionated defaults are present here and absent there.
- **Open-source vs. proprietary, including the pipeline graph.** Palantir is proprietary, and Pipeline Builder is the canonical example of a proprietary orchestration layer customers cannot extract. This architecture uses Dagster (open source) as the pipeline orchestrator, with assets defined in the customer's own repository and run storage in the customer's own Postgres. Foundry customer lock-in is famous; the Dagster choice is a deliberate counter to it.
- **Transparent pricing.** This platform's pricing is published; Palantir's is bespoke.

Where to punt to Palantir:

- Multi-domain enterprises ($5M+ annual platform budget) wanting one substrate for quant + operations + analytics.
- Government, defence, or regulated-industry customers where Palantir's compliance posture is a prerequisite.
- Existing Foundry customers wanting to add quant workflows.

15.4 Databricks

Databricks is the dominant enterprise data platform of the era, with deep investment in Lakehouse architecture, Spark, Delta Lake, and managed MLflow (which they own). They have an explicit "Lakehouse for Quantitative Research" positioning and have invested in financial-services-specific tooling.

Where Databricks wins:

- **Data-engineering scale.** A petabyte-scale data lake with Spark on top is exactly what Databricks is built for. A quant shop with massive alt-data volumes (satellite imagery, social-media firehoses, 13F replication on a multi-year window) is a natural Databricks customer.
- **MLflow ecosystem.** Databricks owns MLflow; their managed offering is the most mature MLflow deployment available.
- **Broad enterprise sales motion** with mature procurement relationships at most large institutions.
- **Spark / Delta Lake** is a real moat for workloads that need them.

Where this architecture wins:

- **Right-sized for actual quant data volumes.** Postgres (with TimescaleDB extension for time-series) handles the operational data volume of a $5B AUM hedge fund trivially. The customer does not need Spark; running Spark for daily-bar equity data is overkill. The simpler architecture is faster, cheaper to operate, and easier for small teams to reason about.
- **Quant-native vs. generic ML.** Databricks's MLflow is the same MLflow; this architecture's discipline (walk-forward enforced, PBO/DSR computed, bi-temporal data, asset checks at every medallion boundary via Dagster) is layered on top. A Databricks customer adding this discipline is doing the integration work themselves. Databricks Workflows is a credible orchestrator within the Databricks ecosystem; outside it, asset definitions do not travel. Dagster's asset definitions live in the customer's repository and are portable across deployments.
- **Silo + BYOC vs. workspace model.** Databricks workspaces are managed multi-tenant entities; not the right shape for the security-conscious hedge fund customer.
- **Opinionated vs. toolkit.** Databricks is a powerful toolkit; this is a focused product. For a customer who knows what they want to build, the toolkit is great; for a customer who wants the best-practice path of least resistance, the focused product is faster.

Where to punt to Databricks:

- Petabyte-scale customers with alt-data-heavy workloads where Spark is genuinely required.
- Customers already on Databricks who want to add quant workflows.
- Customers whose team has Spark expertise and prefers the toolkit model.

15.5 WorldQuant Brain and QuantConnect

These are not direct competitors but are sometimes mentioned in the same conversations. WorldQuant Brain is a crowdsourced-alpha platform where contributors submit alpha factor candidates for evaluation against WorldQuant's proprietary capital. QuantConnect is a retail-focused algorithmic-trading platform with backtesting and live-trading capabilities for individual quants and small funds.

Neither is a competitor to the platform this blueprint describes. WorldQuant Brain is a sourcing channel for WorldQuant's own capital, not a productionalization platform for an external manager. QuantConnect's customer is an individual or a $50M-AUM small fund, several segments below the mid-market hedge funds this platform targets.

The honest framing in a sales conversation: "WorldQuant Brain is interesting if you want to submit your alpha for WorldQuant to trade their capital on. QuantConnect is interesting if you are a retail systematic trader. Neither addresses what you need if you are a $1B AUM hedge fund running your own capital with LP allocators asking for ODD evidence."

15.6 The build-your-own option

For most platform-vendor sales conversations, the most frequent competitor is "we are thinking about building it ourselves." This is the comparison that requires the most honesty, because it is the comparison the customer is most likely to defend internally.

Where build-your-own wins:

- **Total control.** The customer decides every component, every workflow, every UI choice.
- **No vendor relationship to manage.** No procurement cycle, no contract negotiation, no annual renewal pressure, no risk of price increases.
- **No recurring license cost.** A capitalised build amortises against years of avoided licensing.
- **Customisation.** The build can be tuned exactly to the team's existing workflow, existing data sources, existing tools.
- **Engineering knowledge retention.** Building it builds expertise; the team owns the substrate they depend on.
- **Cultural fit.** Many quant shops have engineering cultures that prefer building over buying as a default.

Where this architecture wins:

- **Time-to-value.** This platform is operable in weeks; a competent in-house build to reach feature parity takes 18-30 months minimum, and most attempts do not reach parity at all.
- **Opportunity cost.** Engineering time spent building platform infrastructure is engineering time not spent on alpha-relevant work. Quant shops typically have engineering shortages, not engineering surpluses.
- **Best-practice baseline.** This platform encodes the López de Prado measures, the bitemporal data discipline, the research-prod parity pattern, the audit-chain implementation. An in-house build needs to discover or rediscover these. Some teams will; many will not.
- **Compounding.** Every feature added to this platform is a feature the customer gets without spending engineering on it. An in-house build receives only what its own engineering team adds.
- **Hire leverage.** A new quant hire is productive on this platform in week one. A new quant hire on a custom internal stack is productive in month three.

Where to acknowledge the build-your-own argument is right:

- **$20B+ AUM funds with 50+ engineers.** They have the capacity to build well. They typically prefer to build. We do not target them as customers; if their build wraps, we are a Phase-3 component supplier.

- **Funds with engineering teams that have just been authorised to build a platform.** Political momentum is hard to redirect; do not try.
- **Funds with extreme customisation needs** (highly unusual data sources, non-standard model lifecycle, regulatory requirements specific to their domicile). They build because they have to; we are not the right product for them in v1.

The honest sales line: "Building this yourself is a defensible choice if you are a $20B+ fund with 50+ engineers and a CTO with two years of patience. For a $500M-$5B fund with under 25 engineers, the maths does not work: 18 months of platform-engineering work that we provide on day one is a strictly better outcome than 24 months of partial completion that you maintain forever."

15.7 Jenny Lin's platform (peer competitor)

The user's own memory notes Jenny Lin as a peer founder building a parallel hedge-fund quant productionalization platform. Without specific public information about Jenny's product, the comparison is necessarily provisional.

Probable shape of the comparison:

- The space is large enough to support multiple platform vendors. Even if Jenny's product is similar, the early market is sales-velocity-constrained, not product-similarity-constrained.
- The differentiators that hold against established competitors (silo + BYOC, open-source-first, quant-native opinionated defaults) likely also hold against Jenny's platform unless Jenny made identical choices on all three. The probability of identical choices on all three is low.
- The relationship is currently warm and may remain so; it is not zero-sum.

Approach in conversation: maintain warm relationship; sharpen differentiation along the three load-bearing axes; be prepared for either independent competition or eventual partnership; do not assume hostile dynamics until evidence supports them.

15.8 Summary table

Competitor	Their best fit	Our best fit
SigTech	$5B+ AUM, multi-tenant tolerated, enterprise budget	$500M-$5B AUM, silo/BYOC required, mid-market budget
Domino	Multi-domain enterprise data-science teams	Specifically quant teams who want quant-native defaults
Palantir Foundry	Multi-domain enterprises with $5M+ platform budgets	Mid-market quant shops with self-serve sales motion
Databricks	Petabyte-scale data lakes with Spark workloads	$500M-$20B AUM where operational data fits in Postgres
WorldQuant Brain	Individual contributors selling alpha to WorldQuant	(no overlap)
QuantConnect	Retail systematic traders	(no overlap)
Build-your-own	$20B+ AUM with 50+ engineers	$500M-$5B AUM with under 25 engineers

Competitor	Their best fit	Our best fit
Jenny Lin's platform	(insufficient public information)	(insufficient public information)

15.9 What this chapter implies for the architecture

A reader who has worked through this chapter should now understand why the blueprint makes the specific architectural choices it does:

- **Silo tenancy** is chosen because the SigTech-style multi-tenant model is the most common credible alternative and we are deliberately differentiating against it.
- **Open-source-first stack** is chosen because every credible competitor is proprietary and we are deliberately differentiating against that pattern.
- **Postgres-centric** is chosen because the customer's operational data volume does not require Spark and the simpler substrate is a deliberate non-Databricks choice.
- **Local-first development** is chosen because the SigTech / Domino / Palantir / Databricks model of cloud-tethered research is slower and we are deliberately optimising the dev loop.
- **CQRS event log + cryptographic audit** is chosen because the build-your-own and the proprietary-vendor alternatives both produce inferior audit posture and we are deliberately making this a default rather than an integration project.

The architectural choices in chapters 5-14 are *legible as anti-positioning* once this chapter has been read. That is the intent.

Chapter 16

Build Sequence

16.1 Principle: sequence, not schedule

The usual reflex in a blueprint is to propose a calendar — weeks per phase, milestones at months, a Gantt chart. That reflex assumes human implementation speed, and it does not survive contact with an agentic development environment in which Claude Code is the primary implementor. The constraint is no longer engineer-hours. The constraints are prerequisite order (what must exist before what), architectural integrity (decisions made in one phase that must not be unmade in a later phase), and feedback loops (what must be testable before the next phase can be planned with confidence).

This chapter describes the build sequence as a directed graph of work, with each phase defined by its prerequisites and its acceptance condition rather than by elapsed time.

16.2 Phase 0 — foundations

Acceptance condition: a fresh repository checkout runs end to end locally via a single Makefile target, with authentication, a representative command path, a representative query path, and a smoke training run all exercising real code against local containers. CI executes the same flow against the same container definitions and returns a green status.

Scope:

- Repository structure: Python package layout, React frontend scaffold, Makefile target vocabulary, Dockerfile, docker-compose stack with Postgres (all extensions), MinIO, MLflow, Dagster (run storage in the same Postgres), mock OIDC
- Alembic migration scaffold with an initial schema covering events, aggregates, PGMQ queues, user and role tables, audit log
- Seed data fixtures (parquet files and SQL inserts) representing a working tenant
- FastAPI application skeleton: auth router, one placeholder command endpoint, one placeholder query endpoint, health check, OpenAPI generation
- React SPA skeleton: login flow, authenticated shell, route structure matching the API
- Structured logging, OpenTelemetry metrics and tracing instrumentation
- CI/CD pipeline: PR checks, main workflow, container build and publish to Artifact Registry, Workload Identity Federation
- Terraform module for a tenant project (provisioning of Cloud Run, Cloud SQL, GCS, Secret Manager, IAM)

Prerequisites: none. This phase is entered from a blank slate.

Why first: the foundations establish the test harness against which every subsequent phase will be validated. Phase 0 is not customer-visible; it is the scaffolding that makes every later phase fast and safe. Shortening this phase is a false economy.

16.3 Phase 1 — minimum viable platform

Acceptance condition: one real customer performs one real workflow end to end — authenticate, ingest a data file, train a model, promote it, invoke inference, inspect the audit trail — against a production deployment.

Scope:

- OIDC federation with the first customer's identity provider (Google Workspace or Entra)
- Role-based authorisation with an initial role vocabulary (quant, risk, admin, viewer)
- File-based ingestion for one source (SFTP pull or GCS drop), bronze landing, silver transformation, gold aggregation
- Model training orchestration with a single compute target (Cloud Run Job) and MLflow integration
- Synchronous model serving for one registered model through the REST API
- Point-in-time correctness enforced in the training data extraction path
- React UI screens covering the golden-path journey
- Playwright end-to-end coverage of the golden-path journey
- First production tenant provisioned by an engineer running `terraform apply` directly against the per-tenant module and live (the control plane that automates this provisioning flow is introduced in Phase 5)

Prerequisites: Phase 0 complete.

Why this scope: the goal is to exercise every architectural layer in production — authentication, data platform, model training, model serving, audit — rather than to ship a feature-rich product. Every subsequent phase adds depth to layers already proven. A feature missed in Phase 1 is cheap to add later; an architectural gap discovered in Phase 5 is expensive.

16.4 Phase 2 — training depth

Acceptance condition: the platform supports the realistic compute profiles of the target customer segment, including GPU training and walk-forward validation that gates model promotion.

Scope:

- GPU training dispatch through Vertex AI Custom Training, using the same training orchestration surface
- Walk-forward validation harness that replays historical dates with strict point-in-time data and produces out-of-sample performance metrics
- Model validation gates that block promotion on failed statistical tests, backtest underperformance, or compliance rule violations
- Hyperparameter search coordination with MLflow comparison views
- Model lineage: every trained model is linked to its training dataset snapshot, its code commit, its validation report

Prerequisites: Phase 1 complete; a customer with a training workload exceeding Cloud Run Job limits.

16.5 Phase 3 — data pipeline breadth

Acceptance condition: the platform ingests data from the full set of source types common to the target customer segment and exports results in the formats those customers demand.

Scope:

- File-based ingestion for the full source matrix: SFTP pull, GCS push drop, HTTPS API pull, scheduled vendor API calls
- File-based export: scheduled output files to customer-accessible GCS buckets, SFTP push to customer endpoints, on-demand signed URL downloads
- Per-source data quality validation with quarantine routing and configurable thresholds
- Bi-temporal data pattern applied across silver and gold tables
- Reconciliation jobs producing automated daily comparisons between bronze, silver, and gold layer row counts and key aggregates
- Backfill tooling — a CLI and a minimal UI — for re-running pipeline ranges after source corrections

Prerequisites: Phase 1 complete; identification of at least three distinct source types across existing customers.

16.6 Phase 4 — serving breadth

Acceptance condition: the platform supports the three inference modes (synchronous, batch, scheduled) with appropriate quality-of-service for each, and supports model version promotion without customer-perceptible disruption.

Scope:

- Batch inference endpoints consuming files or dataset references and producing GCS outputs
- Scheduled inference pipelines (e.g. end-of-day scoring, overnight risk calculation) integrated with APScheduler
- Dedicated Cloud Run services for high-QPS or GPU-bound inference, auto-routed from the main application
- A/B traffic splitting between model versions for champion-challenger testing
- Canary inference — new model versions serving a small traffic fraction with statistical comparison before full promotion
- Inference audit trail queryable from the UI and exportable on demand

Prerequisites: Phase 1 complete; a customer with serving requirements beyond low-QPS synchronous inference.

16.7 Phase 5 — fleet operations

Acceptance condition: adding a new tenant is an operator action rather than an engineering project, and upgrading the fleet to a new release is an orchestrated process that runs unattended within defined safety bounds.

Scope:

- Control plane application with tenant registry, provisioning dashboard, and per-tenant telemetry

- Automated tenant provisioning — operator fills a form, control plane drives Terraform, tenant is live within its SLA
- Automated upgrade orchestration with wave-based rollouts (canary, early-adopter, general, conservative) and per-tenant maintenance windows
- Per-tenant customer-accessible dashboards (Cloud Monitoring IAM grants plus optional embedded views in the UI)
- Per-tenant cost attribution pulling from BigQuery billing export
- BYOC support: Terraform module executed under customer-granted service accounts, customer-owned state buckets, documented IAM grant template

Prerequisites: Phases 1, 3, and 4 complete; at least three tenants in production to expose fleet-management friction that is invisible with one tenant.

16.8 Phase 6 — compliance surface

Acceptance condition: the platform carries the certifications and demonstrable controls required by the contractually-demanding segment of the target market.

Scope:

- SOC 2 Type II readiness: control definitions, evidence collection automation, external audit preparation
- Cryptographic chaining of the audit trail and tamper detection job
- WORM retention configuration for audit and inference logs with Object Lifecycle Lock
- Self-service export endpoint producing signed, timestamped archives for regulator requests
- CMEK (Customer-Managed Encryption Keys) support for tenants demanding it
- Additional regional deployments (UK, EU, APAC) as driven by customer demand
- Formal penetration testing cadence and remediation workflow

Prerequisites: Phase 5 complete; a customer whose contract terms require any of the above.

16.9 Dependency graph

The phases are not strictly linear. Phase 2 (training depth) and Phase 3 (pipeline breadth) are independent — either may be entered immediately after Phase 1, driven by whichever real customer need surfaces first. Phase 4 (serving breadth) depends on Phase 1 but not on Phase 2 or Phase 3. Phase 5 depends on operational evidence that can only be obtained after at least Phases 1, 3, and 4 are exercised against multiple customers. Phase 6 is gated by customer demand and by the cumulative maturity of Phases 1 through 5.

A project that optimises purely for demonstrable progress rather than architectural integrity is tempted to skip Phase 0 and parallelise Phases 2 through 4. This is the wrong trade. Phase 0 is the test harness; Phases 2 through 4 are features that share the test harness; Phase 5 is the graduation from “product that works” to “service that scales.” Respecting the dependency graph produces a system that stays coherent at Phase 6; ignoring it produces a system that requires a rewrite between Phase 4 and Phase 5.

16.10 What gets deferred, and when to revisit

Earlier drafts of this blueprint deferred a dedicated orchestration framework (Dagster, Prefect, or Airflow) on the grounds that APScheduler plus PGMQ covered the pipeline graph for realistic

customer scale. That deferral has been reversed: Dagster is in v1, used as the data-and-ML asset orchestrator. The decisive factor was the asset-graph lineage UI, which is the visual artefact LP allocators want to see during operational due diligence and which APScheduler plus PGMQ alone cannot produce. Prefect and Airflow remain not pursued; the asset model in Dagster is a better fit for the medallion data shape than Airflow's task graph, and the asset-check surface is the canonical place to encode data-quality rules. The Data Platform chapter documents the integration in full.

The remaining deferrals continue to sit outside the phase plan until specific conditions make them earn their place:

Component	Deferred because	Revisit when
Feature store (Feast)	Quant customers typically embed feature extraction in model code	Train-serve skew becomes a recurring incident source, or cross-model feature reuse is explicitly requested
Kubernetes (GKE)	Cloud Run + Vertex AI covers every computed profile	Customer demands stateful workloads, custom sidecars, or a topology Cloud Run cannot express
Dedicated broker (Kafka / Confluent)	PGMQ handles realistic message rates	Sustained throughput approaches PGMQ's ceiling, or multi-subscriber replay semantics become essential
BigQuery	Postgres + TimescaleDB covers operational analytics	Tick-level data or multi-year backtests require query performance Postgres cannot deliver
Dedicated serving platform (Triton / BentoML / Seldon)	In-process serving covers target latency and throughput	Specific customers require GPU inference at QPS the main application cannot handle

Each deferral is recorded in the repository's `DECISIONS.md` alongside the revisit trigger. When the trigger fires, the component is reconsidered, not automatically added.

16.11 Acceptance discipline

Each phase ends when its acceptance condition is satisfied. The acceptance condition is binary: it is true or it is false. Partial completion is carried forward into the next phase only when the unmet portion does not block the acceptance condition of any subsequent phase. A phase that declares itself complete with unmet acceptance conditions imports debt into the foundation on which every later phase depends; the discipline is to hold the line.

This discipline is especially important in an agentic implementation context, because the temptation to declare completion based on "most of the scope delivered" is amplified by the speed of iteration. The acceptance condition is the ground truth; the scope is the means to the condition, not a substitute for it.